Laboratorio di Elettronica Digitale

- PAL: Programmable Array Logic -

Programmazione a livello basso => si programmano le porte logiche

vanno su applicazioni che costano (civile o militare che sia)

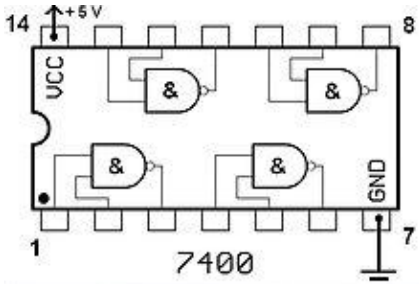
UNIVERSITÀ DEGLI STUDI FIRENZE
DINFO
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

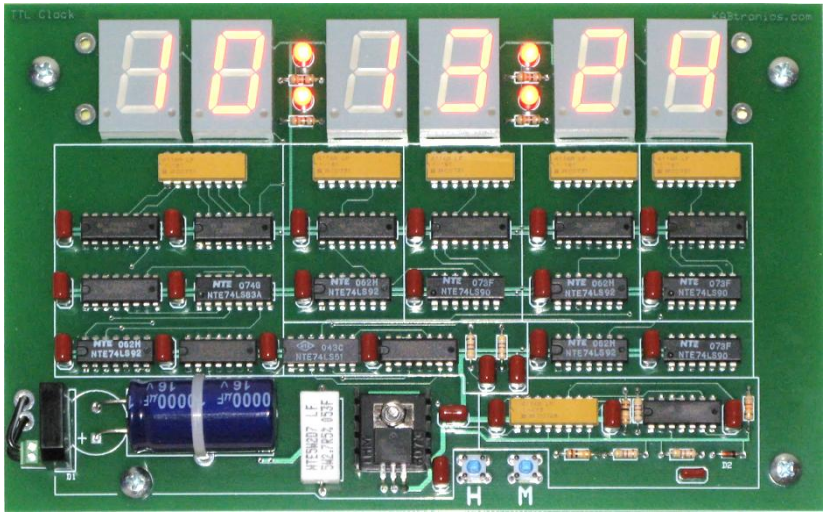
Updated 03/2025

- How logic circuits can be used ?

Circuits exist(ed) that integrates simple logic functions

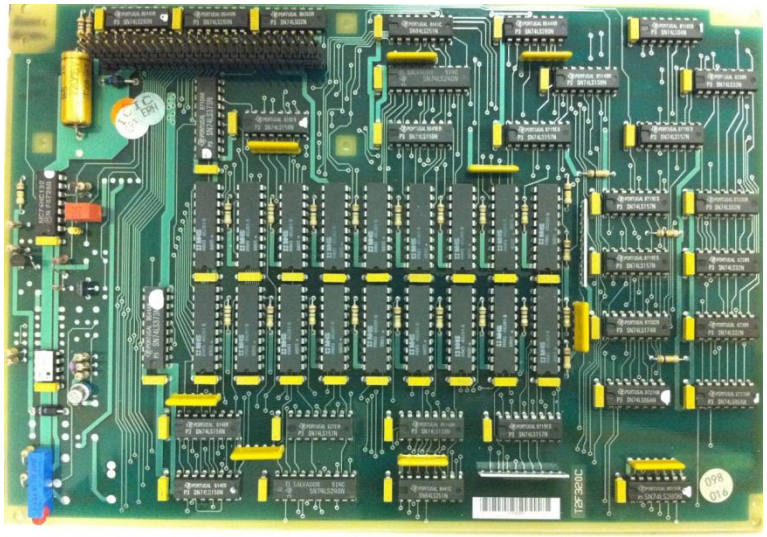
Es. SN 7400 with 4 NAND gates

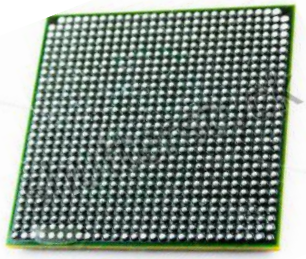




With this solution a simple clock needs a full board!

- In modern circuits all the logics in integrated in a single, programmable chip: a Programmable Logic Device (PLD)



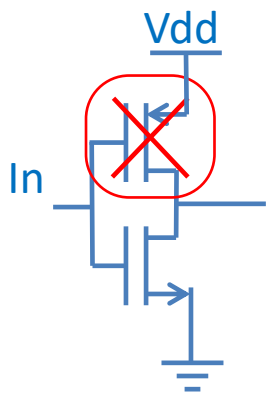


- A single PLD can easily integrate ~ 500.000 gates with 500-1000 pin !

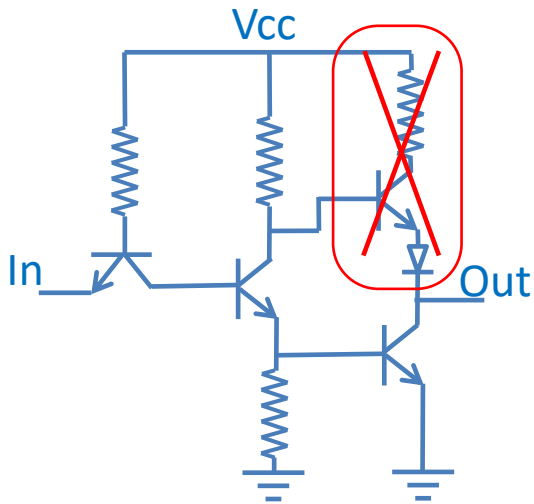
- Let’s try and understand how to use it …

- Open Drain / Open Collector Gates -

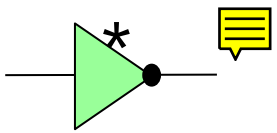
Open Drain NOT CMOS



Open Collector NOT TTL

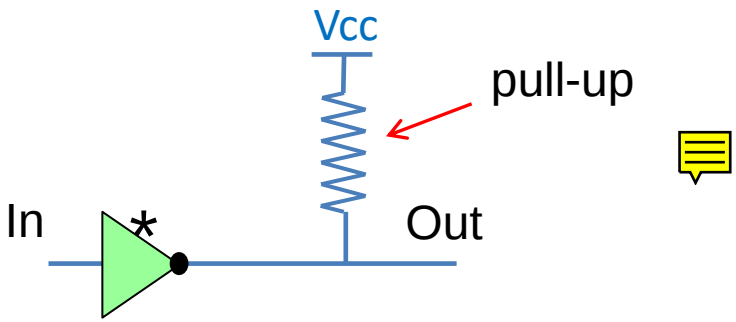


Open drain/collector gates are labelled with a star or similar symbol



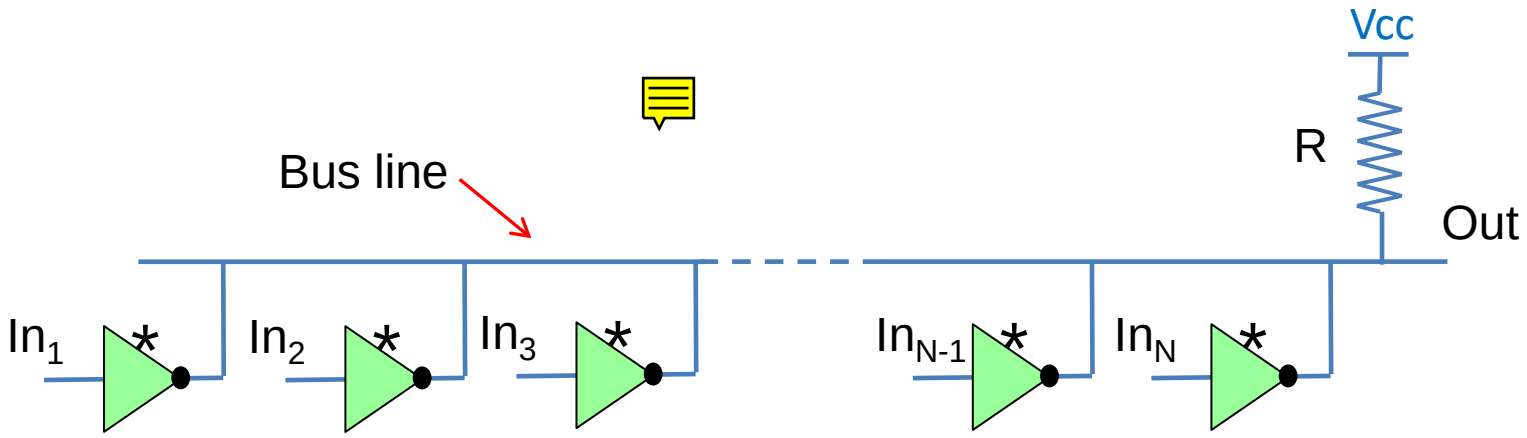
- The «open drain gates are obtained by removing the «high» MOS in CMOS and the «high» transistor-resistor-diode in TTL.
- The gate can only drive low, i.e. sink current from the output to ground, but can't source current from power to the output.
- Note: A Open Drain CMOS NOT gate can be realized with a single NMOS transistor

- Open Drain / Open Collector Gates -



- The gate needs a pull-up to drive the high level
- When Out = ‘0’ the gate shorts the output to ground => 0V in Out
- When Out = ‘1’ the gate is disconnected => Out = Vcc thanks to the pull-up

- Open Drain / Open Collector Gates -

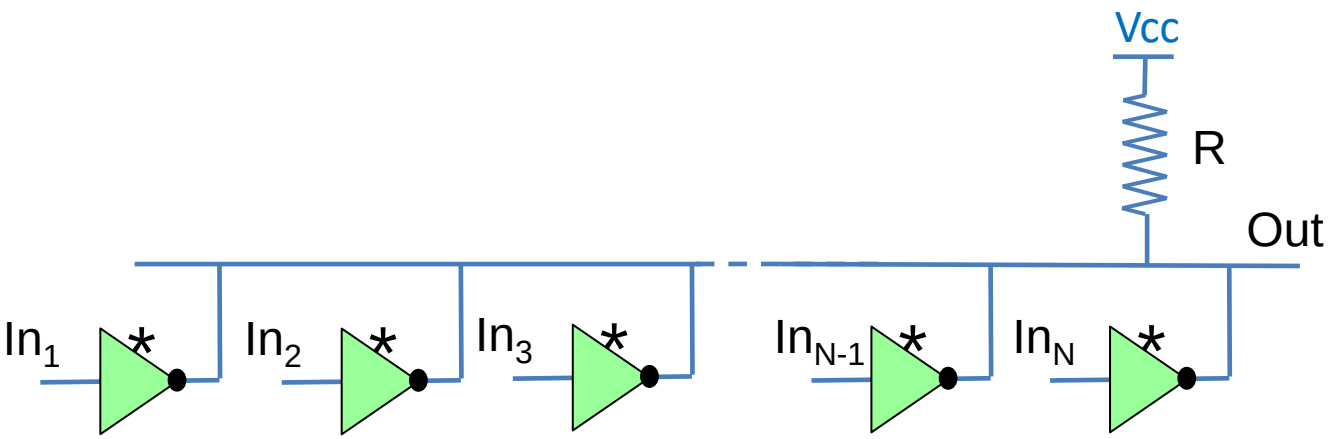


- The output of several gates can be connected on the same bus without risk of bus contention and circuit damage[1]
- In fact, each output or sinks current or is disconnected. The current is sourced to the bus only through the resistor R. => No risk of short

[1]see Elettronica dei Sistemi Digitali course

- Wired Logic -

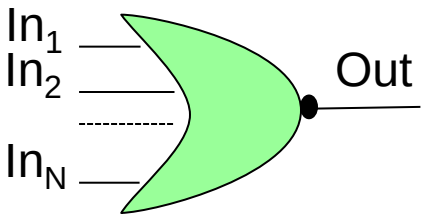
- Open Drain/Collector gates can be used for wired logic



NOR

In_1	In_2	,,,	In_N	Out
0	0	,,,	0	1
0	0	,,,	1	0
0	1	,,,	0	0
0	1	,,,	1	0
1	0	,,,	0	0
1	0	,,,	1	0
1	1	,,,	0	0
1	1	,,,	1	0

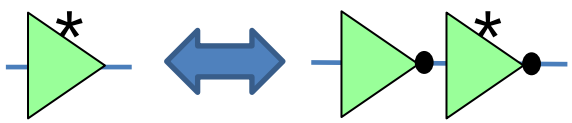
Out is HIGH only if ALL of the N In are LOW (Output disconnected). When one or more In is high, the corresponding output sinks current to ground and the Out goes LOW. This is a NOR gate with N inputs.



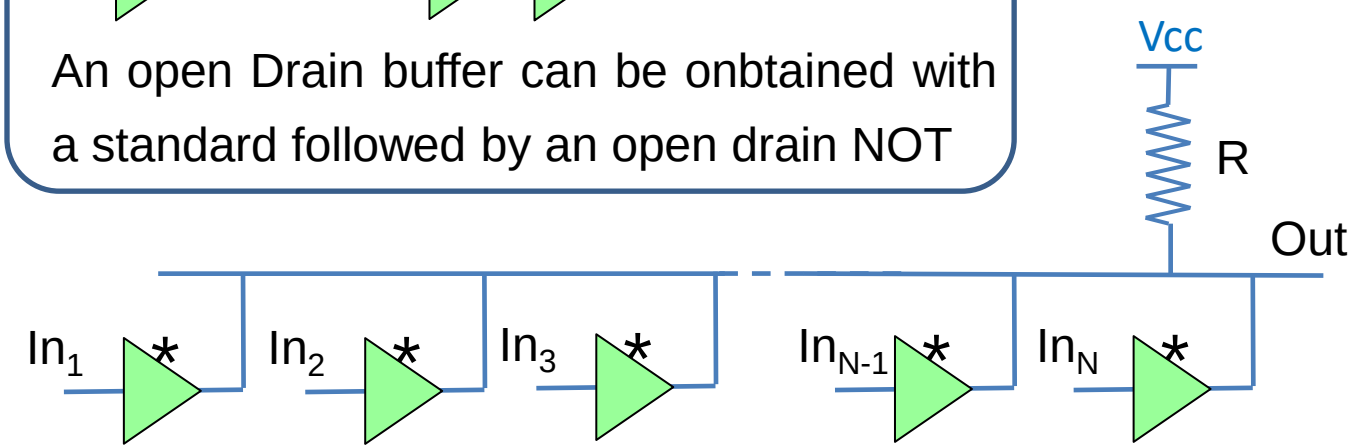
- Wired logic represents a simple way of realizing gates with several inputs

- Wired Logic -

- If we use a buffer instead of a NOT we have a wired AND



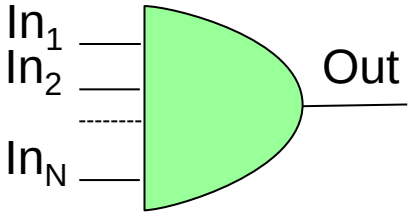
An open Drain buffer can be onbtained with a standard followed by an open drain NOT



AND

In_1	In_2	...	In_N	Out
0	0	...	0	0
0	0	...	1	0
0	1	...	0	0
0	1	...	1	0
1	0	...	0	0
1	0	...	1	0
1	1	...	0	0
1	1	...	1	1

Out is HIGH only if ALL of the N In are HIGH (Principal Mosfet Off). This is a AND gate with N inputs

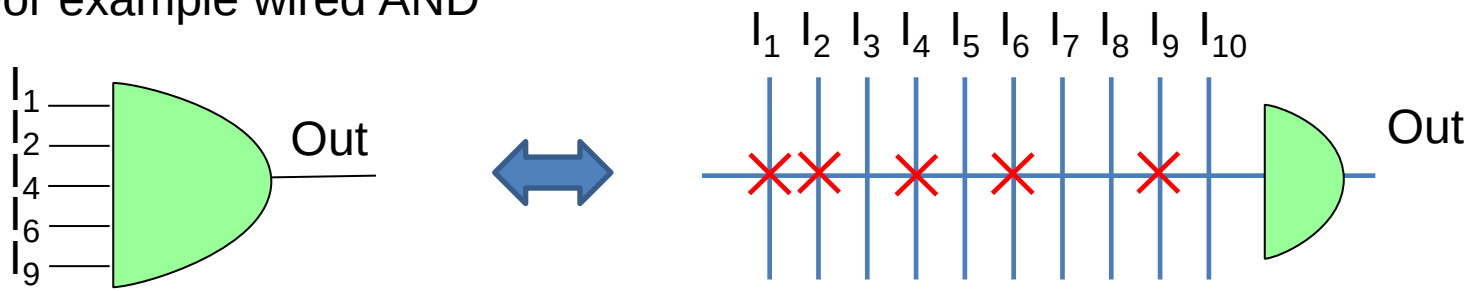


- Programmable Wired Logic -

- Towards a programmable logic…

  A wired logic can be represented as:

For example wired AND

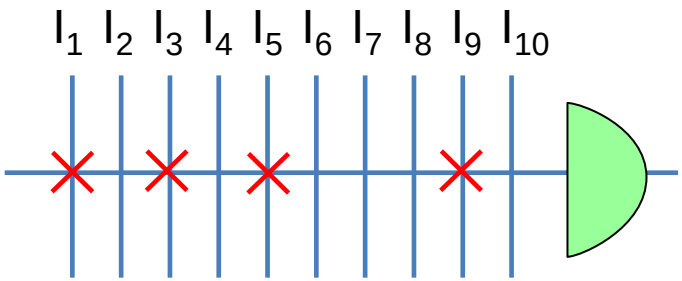


$$Out = I_1 \bullet I_2 \bullet I_4 \bullet I_6 \bullet I_9$$

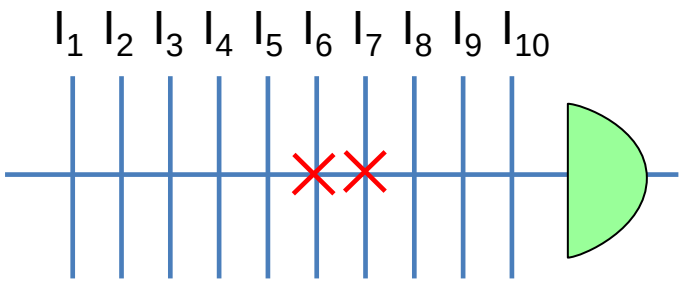
The possible inputs are reported as crossing lines. The inputs actually connected are labelled with a cross. Programming the AND means changing the actual connected inputs (crosses) among the possible inputs $I_{1\text{-}10}$

- Programmable Wired Logic -

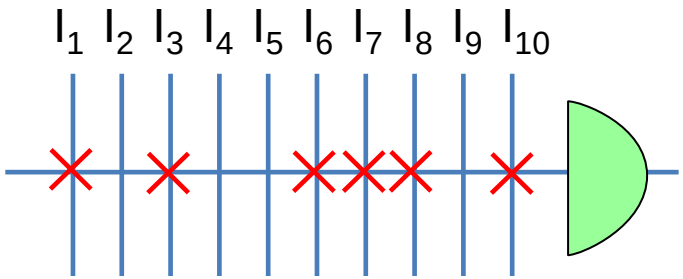
- For example…



$Out = I_1 \bullet I_3 \bullet I_5 \bullet I_9$



$Out = I_6 \bullet I_7$

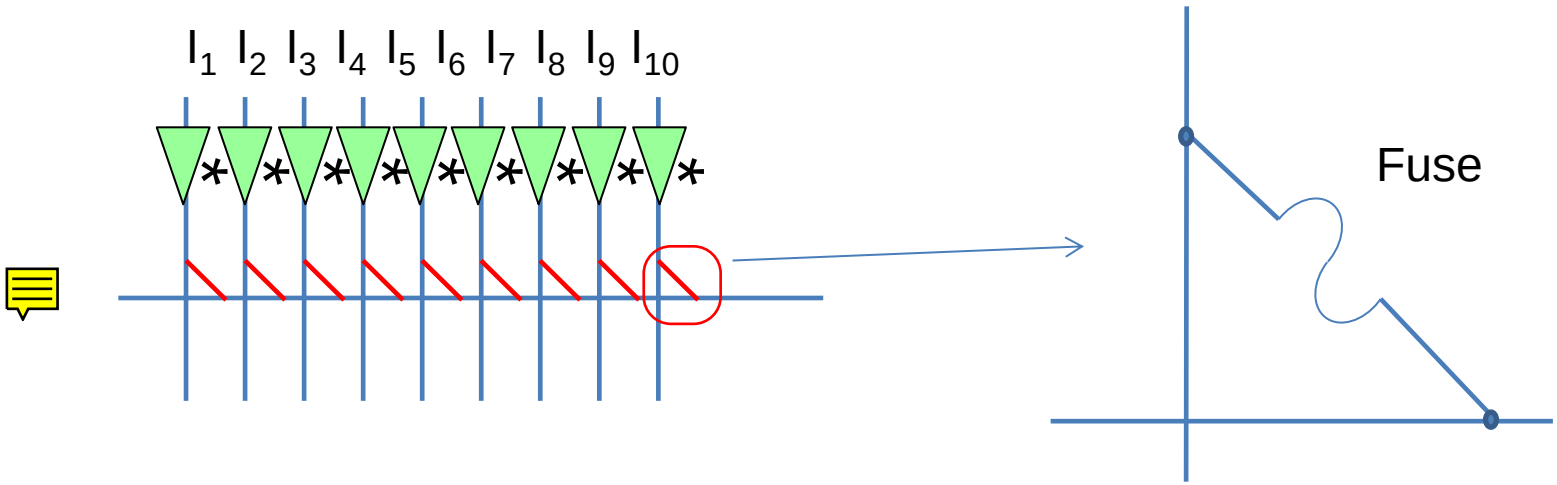


$Out = I_1 \bullet I_3 \bullet I_6 \bullet I_7 \bullet I_8 \bullet I_{10}$

How can this programmability be obtained in a real circuit?

- Programmable Wired Logic -

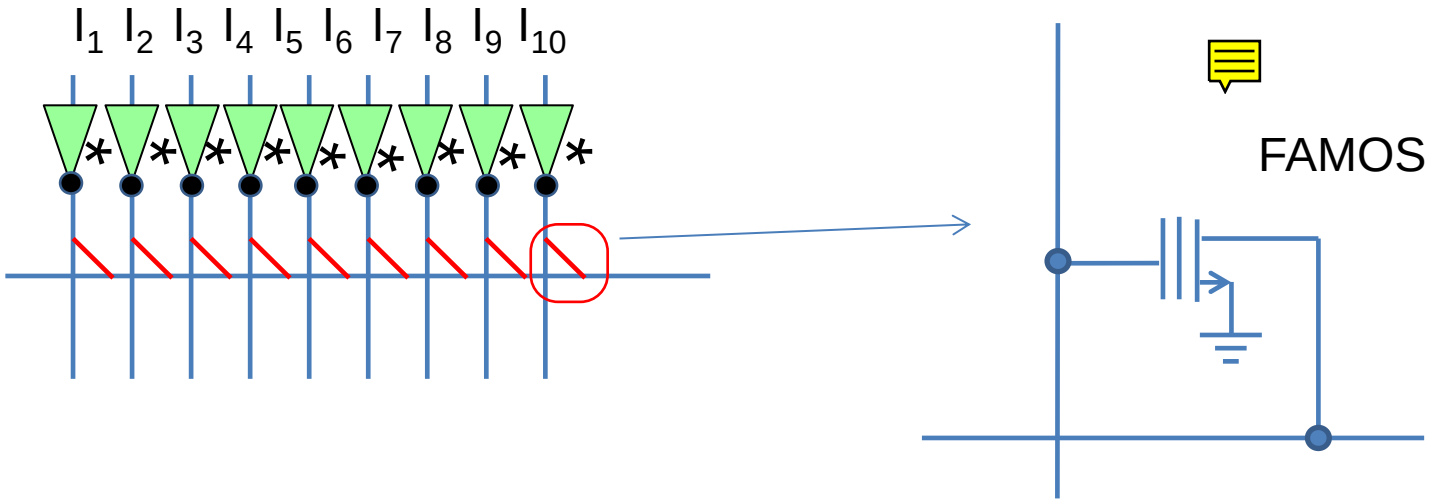
- …By using matallic fuses



A «fuse» is made by a thin metal strip. A high current is used to blow the fuse and remove the connection. Once the connection is removed it can not be restored. These devices can be programmed once (OTP)

- Programmable Wired Logic -

- …By using the Floating gate Avalance injection MOS (FAMOS)

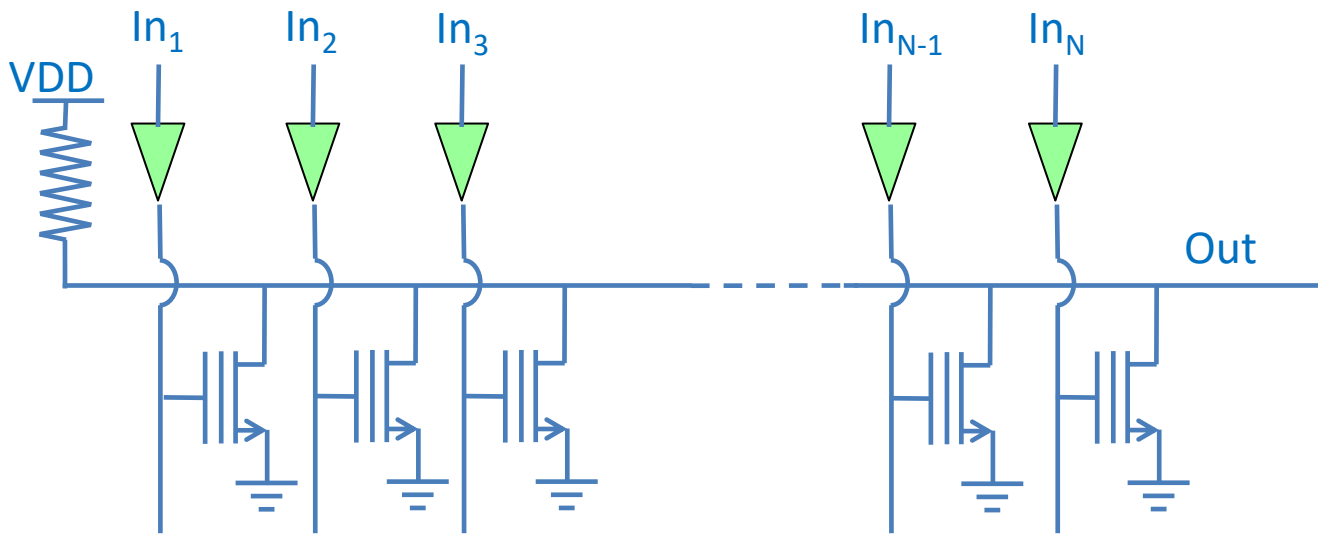


The FAMOS is used also in EEPROM and can be re-programmed several times. When electrons are stored in the floating gate, the FAMOS is an open circuit, when electrons are removed from the floating gate, the FAMOS works like a standard MOS.

Note that in this case we use standard input buffers (NOT in this case), since each NOT is connected just to one FAMOS gate

- Programmable Wired Logic -

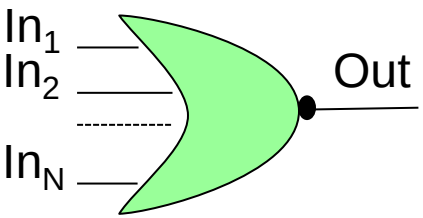
- Programmable NOR with FAMOS



NOR

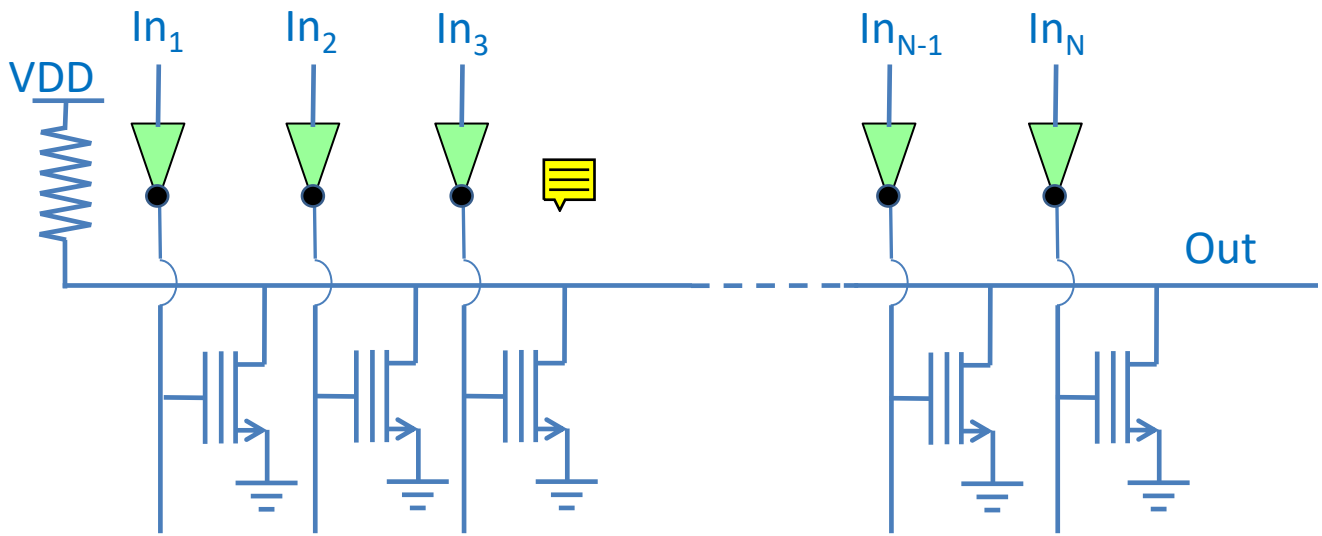
In_1	In_2	,,,	In_N	Out
0	0	,,,	0	1
0	0	,,,	1	0
0	1	,,,	0	0
0	1	,,,	1	0
1	0	,,,	0	0
1	0	,,,	1	0
1	1	,,,	0	0
1	1	,,,	1	0

Out is HIGH only if ALL of the N In are LOW (FAMOS Off). When one or more In is high, the corresponding FAMOS is switched ON, and the Out goes LOW. This is a NOR gate with N inputs. Of course, disabled FAMOS (with electrons on floating gate) are always OFF and the corresponding input does not matter



- Wired Logic -

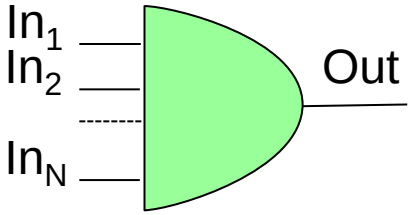
- Programmable AND with FAMOS



AND

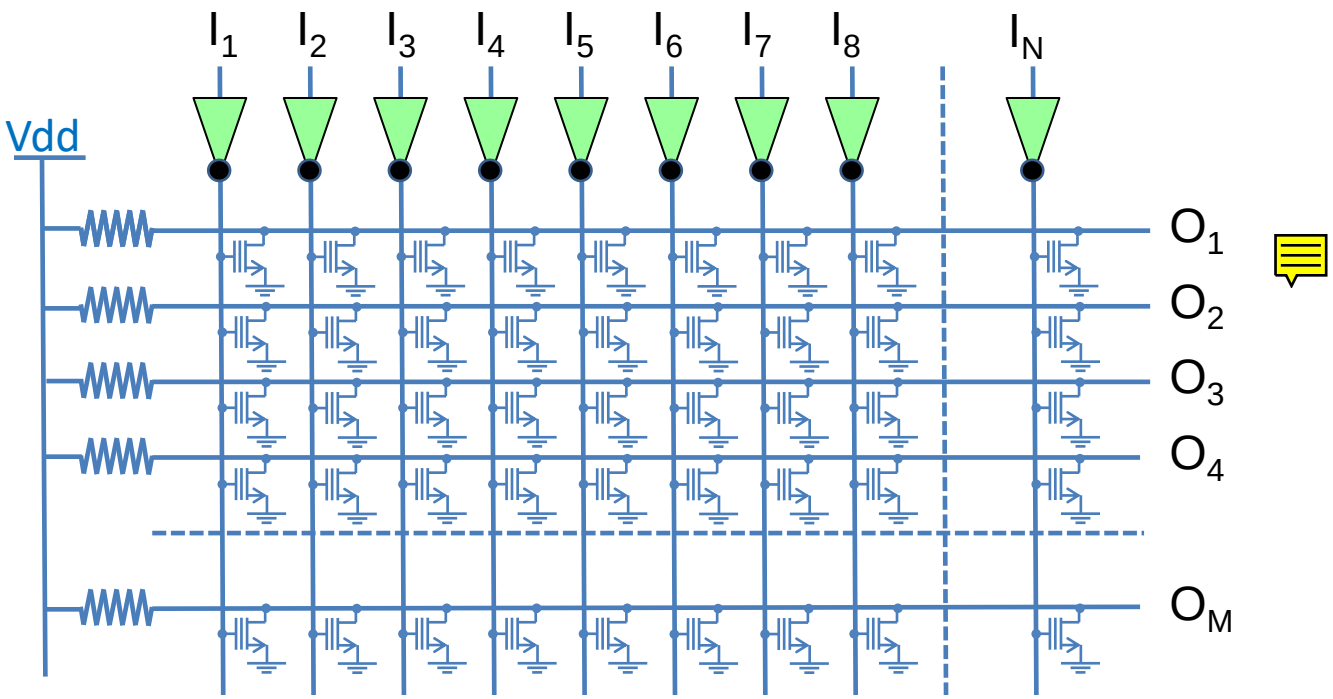
In_1	In_2	...	In_N	Out
0	0	...	0	0
0	0	...	1	0
0	1	...	0	0
0	1	...	1	0
1	0	...	0	0
1	0	...	1	0
1	1	...	0	0
1	1	...	1	1

If a NOT is used as input buffer, we obtain a programmable AND instead of NOR



- Programmable AND Matrixes-

- We can produce a matrix of wired AND as follows

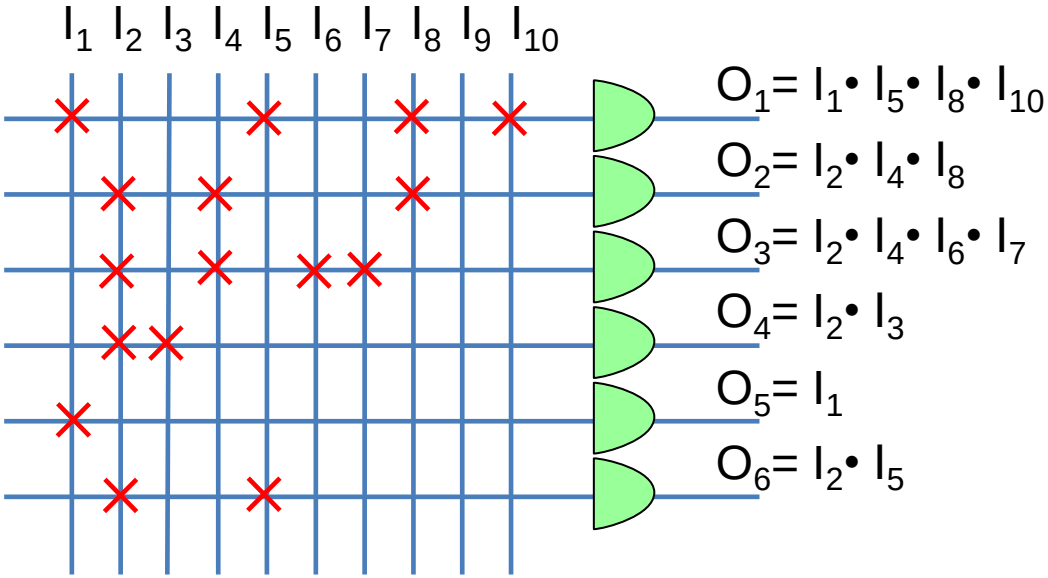


In this example we have a N input – M output programmable AND matrix

The FAMOSes select which inputs are connected to which outputs

- Programmable AND Matrix-

- The matrix is usually represented as :

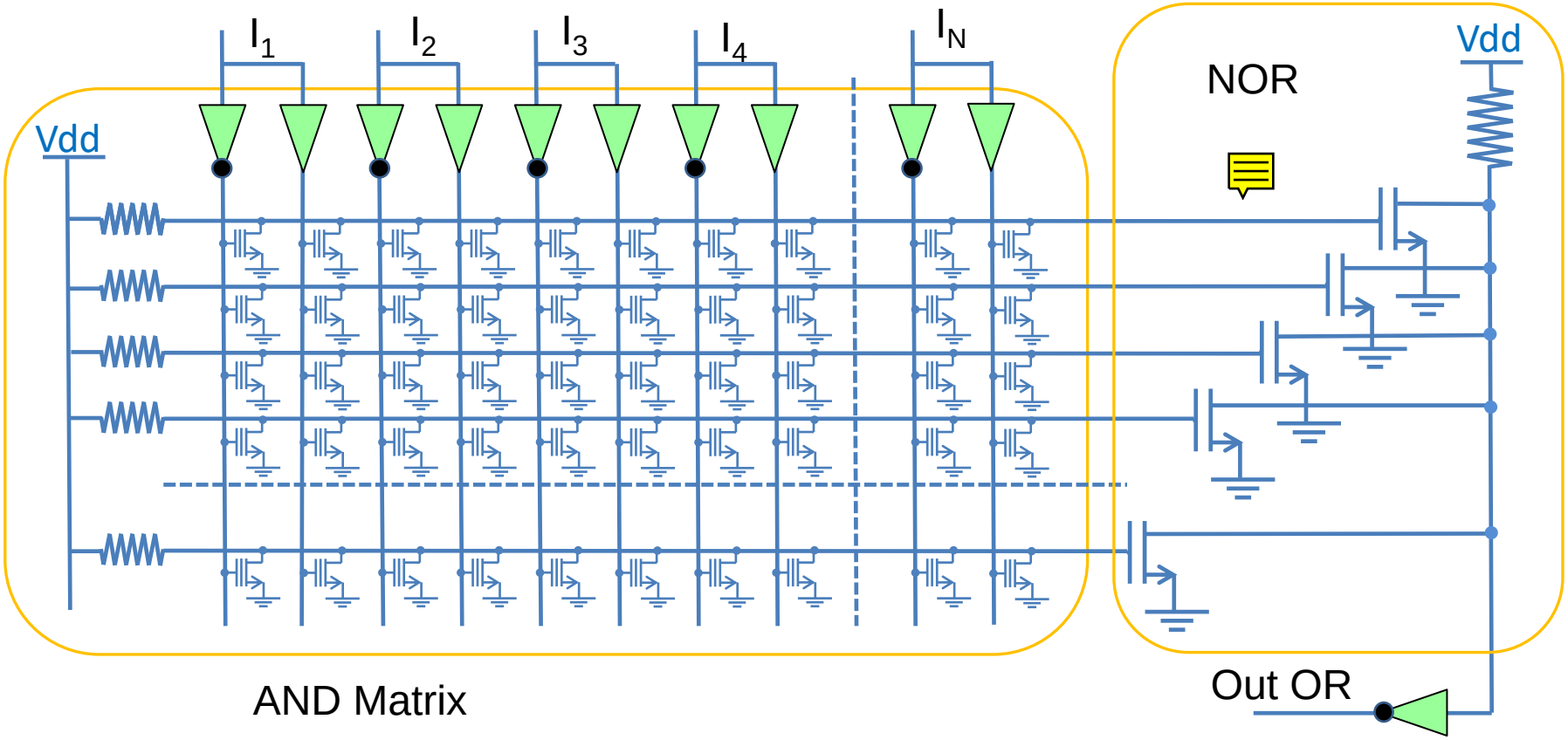


In this example we have 6 wired AND with 10 possible inputs each

Each cross represents a «working» FAMOS, the missing cross a «isolating» FAMOS

- PAL: Programmable Array Logic -

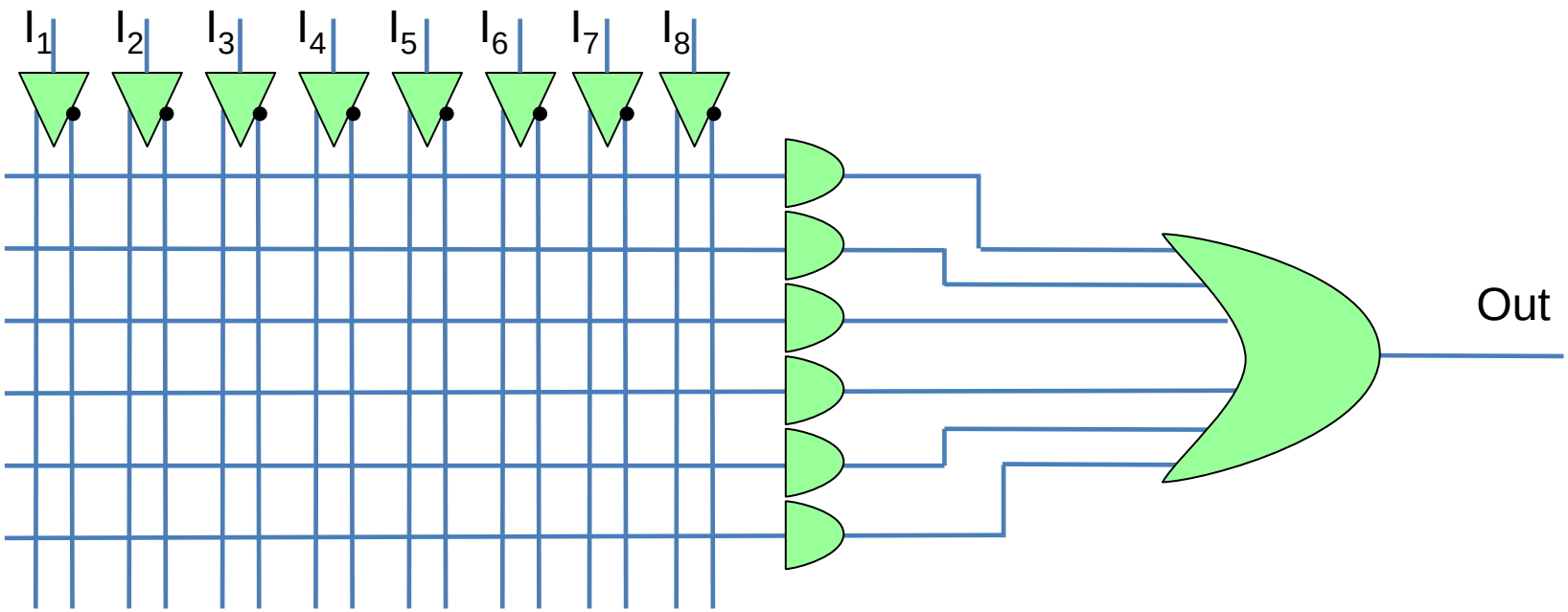
- PLA is realized by a programmable AND matrix and a fixed OR



- The inputs feed the matrix in direct and negate form

- PAL: Programmable Array Logic -

- The PAL is usually represented as :



In this example 8 input feed a programmable AND matrix followed by a 6 input OR

- PAL: Programmable Array Logic -

- Every logic function has a AND-OR representation (2-level).

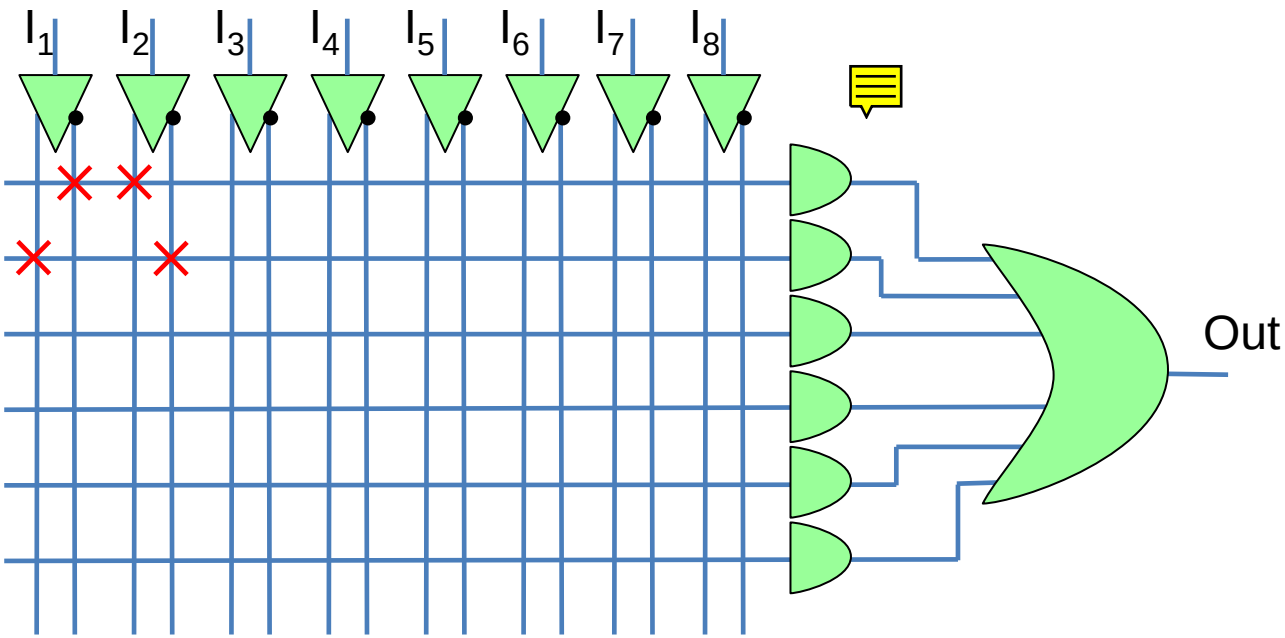
Every logic function can be programmed in a PAL

(provided that the PAL has enough resources)

Example:

I_1	I_2	Out
0	0	0
0	1	1
1	0	1
1	1	0

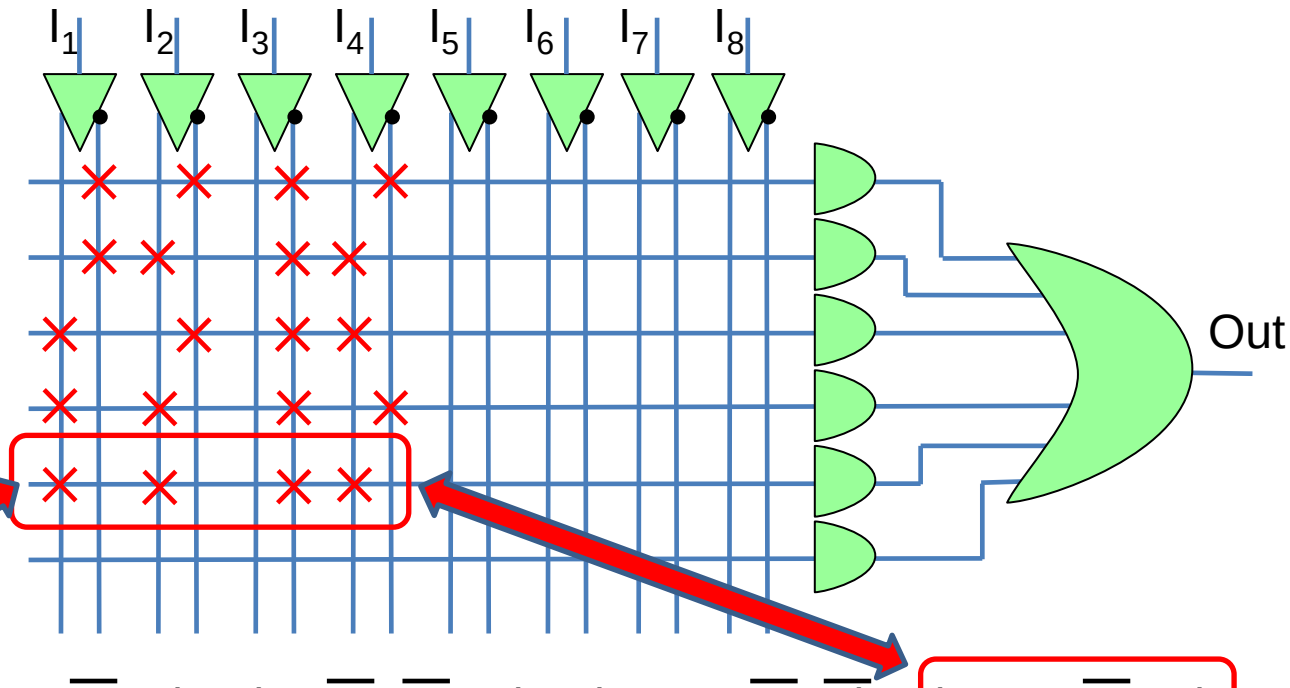
$$Out = (\overline{I_1} \cdot I_2)+(I_1 \cdot \overline{I_2})$$



- PAL: Programmable Array Logic -

Example:

I_1	I_2	I_3	I_4	Out
0	0	0	0	1
0	1	0	1	1
1	0	0	1	1
1	1	0	0	1
1	1	0	1	1
All other combinations				0



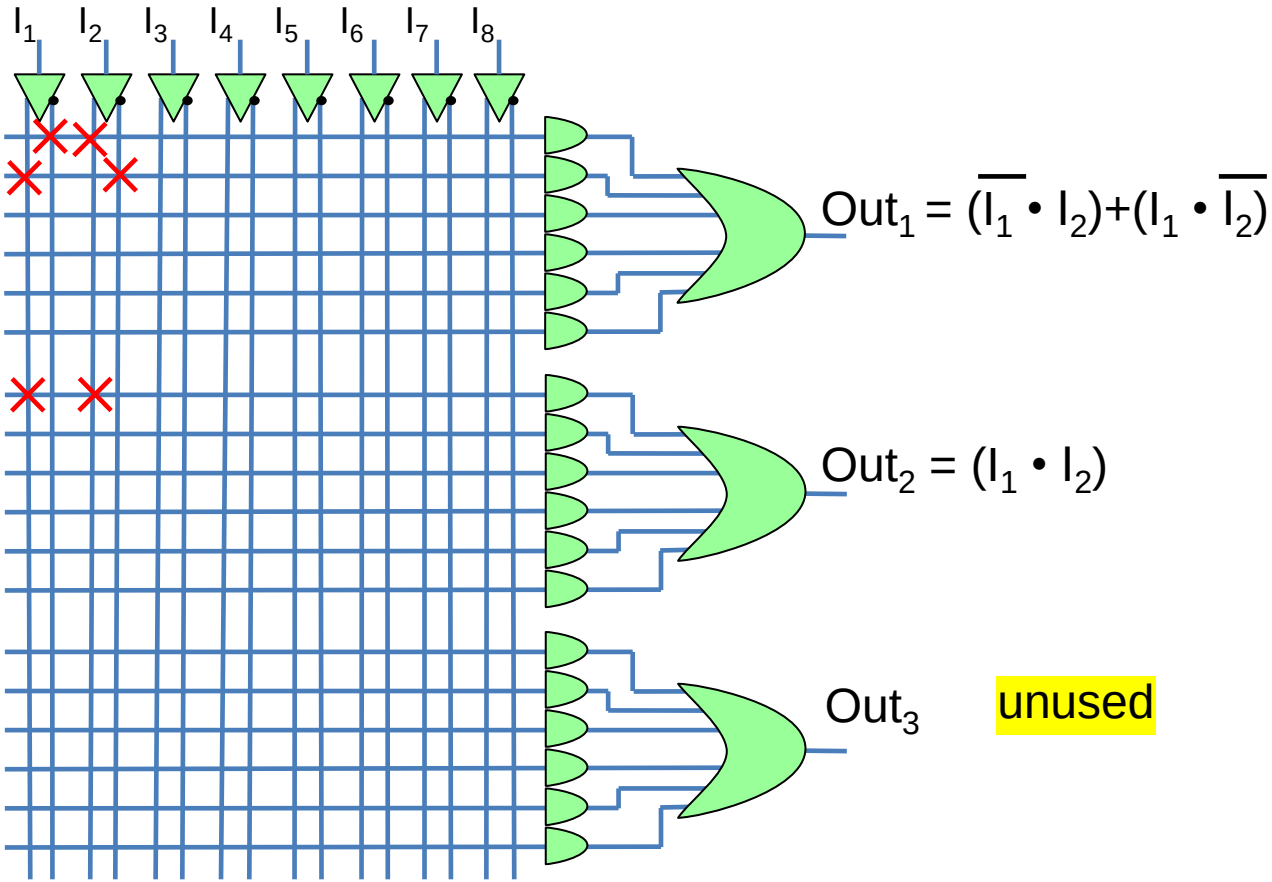
$$Out = (\overline{I_1} \cdot \overline{I_2} \cdot \overline{I_3} \cdot \overline{I_4}) + (\overline{I_1} \cdot I_2 \cdot \overline{I_3} \cdot I_4) + (I_1 \cdot \overline{I_2} \cdot \overline{I_3} \cdot I_4) + (I_1 \cdot I_2 \cdot \overline{I_3} \cdot \overline{I_4}) + \boxed{(I_1 \cdot I_2 \cdot \overline{I_3} \cdot I_4)}$$

Every product term (truth table line) is realized by a wired AND in the PAL

A function with N inputs and M product terms needs a PAL with, at least, N inputs and M lines towards the OR

- PAL: Programmable Array Logic -

- The PAL includes several output (OR gates)



Example:

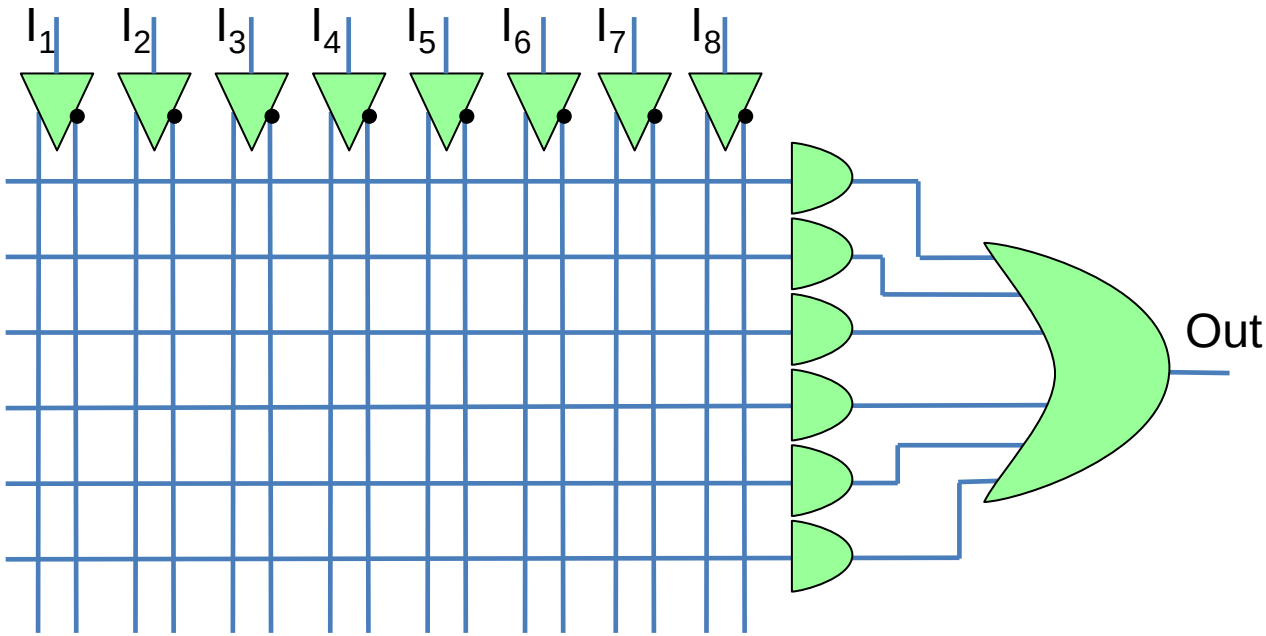
I_1	I_2	Out_1	Out_2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

The inputs (matrix columns) are common to all outputs

- PAL: Programmable Array Logic -

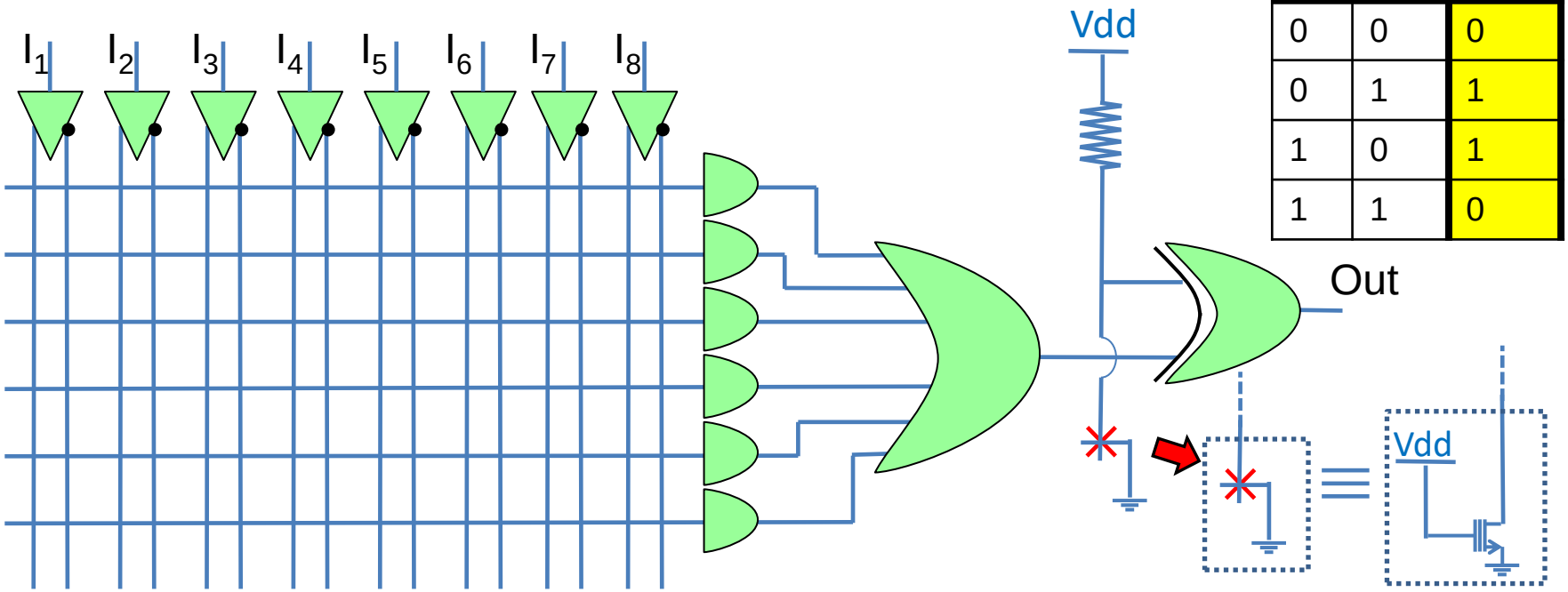
Example:

I_3	I_2	I_1	Out
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



The ‘out’ of this logic funcion has seven ‘1’. As it is, it needs 7 product terms, but this PAL has only 6. This function cannot be realized. However it has only a ‘0’ in output, so $\overline{Out} = I_3 \bullet I_2 \bullet I_1$. The neg function would be very easy to realize….

- PAL: Improvements: Additional XOR -



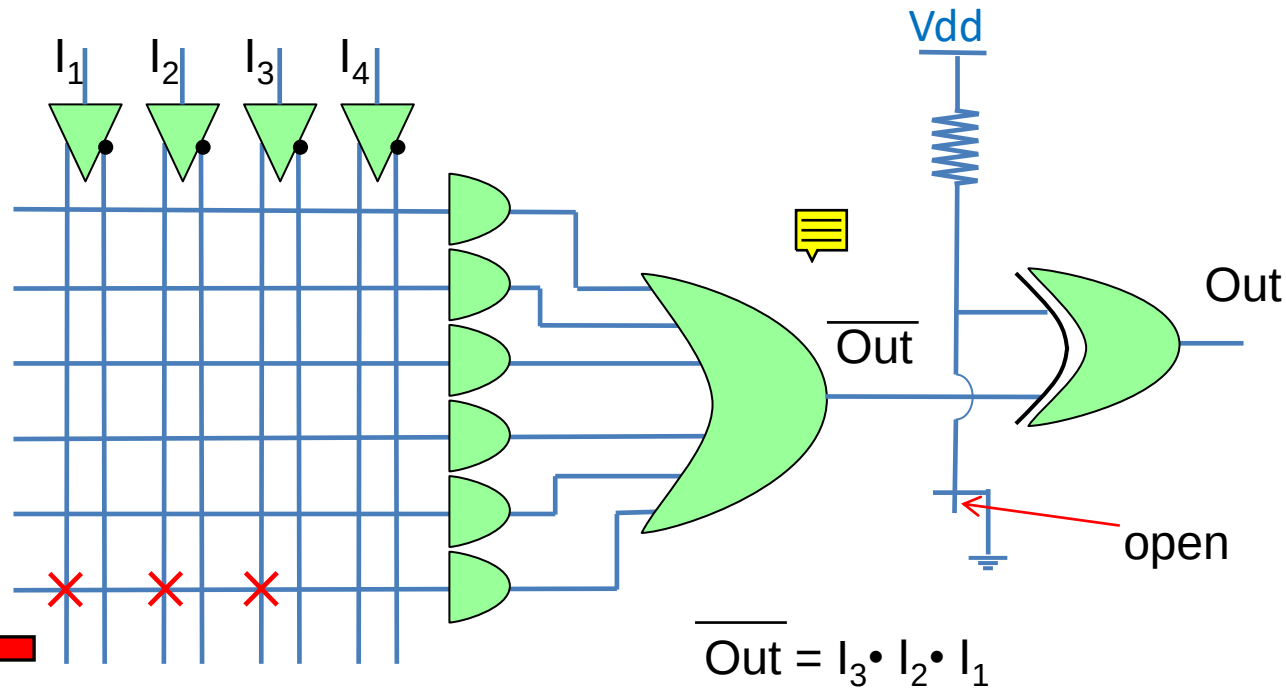
A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The XOR is sort of «programmable NOT». If one input is ‘1’ the other is iverted, if it is ‘0’ the other is not inverted. With a FAMOS we can decide if inverting or not the output. This is useful for the synthesis of function with more ‘1’ than ‘0’ in the output

- PAL: Improvements: Additional XOR -

Example:

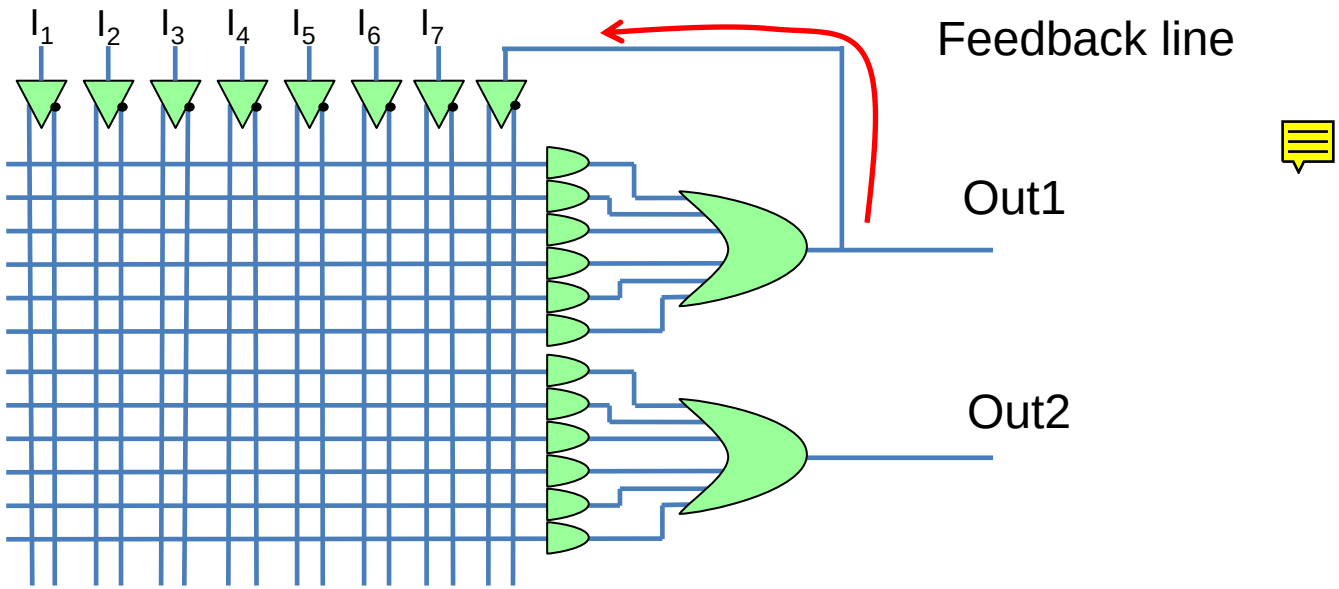
I_3	I_2	I_1	Out	$\overline{Out}$
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1



In the AND matrix the $\overline{Out}$ is coded, then the final XOR inverts the $\overline{Out}$.

The programmable XOR allows to synthesize the ‘1s’ or the ‘0s’ of a logic function, whichever are more convenient

- PAL: Improvements: feedback line -



It is possible to feed an output back in the AND matrix. This allows the realization of more complex functions

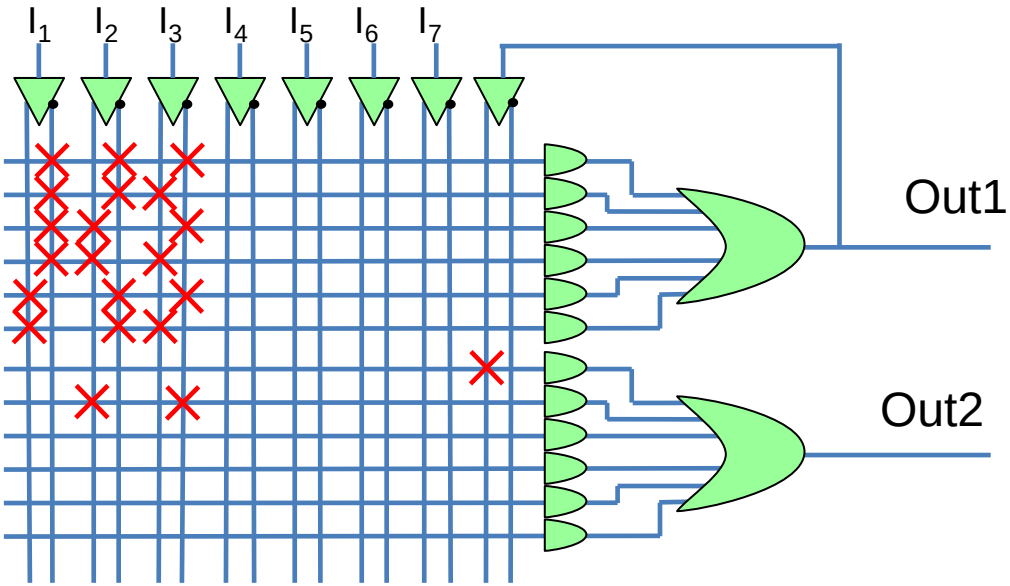
- PAL: Improvements: feedback line -

Example:

I_3	I_2	I_1	Out
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Out1

Out2



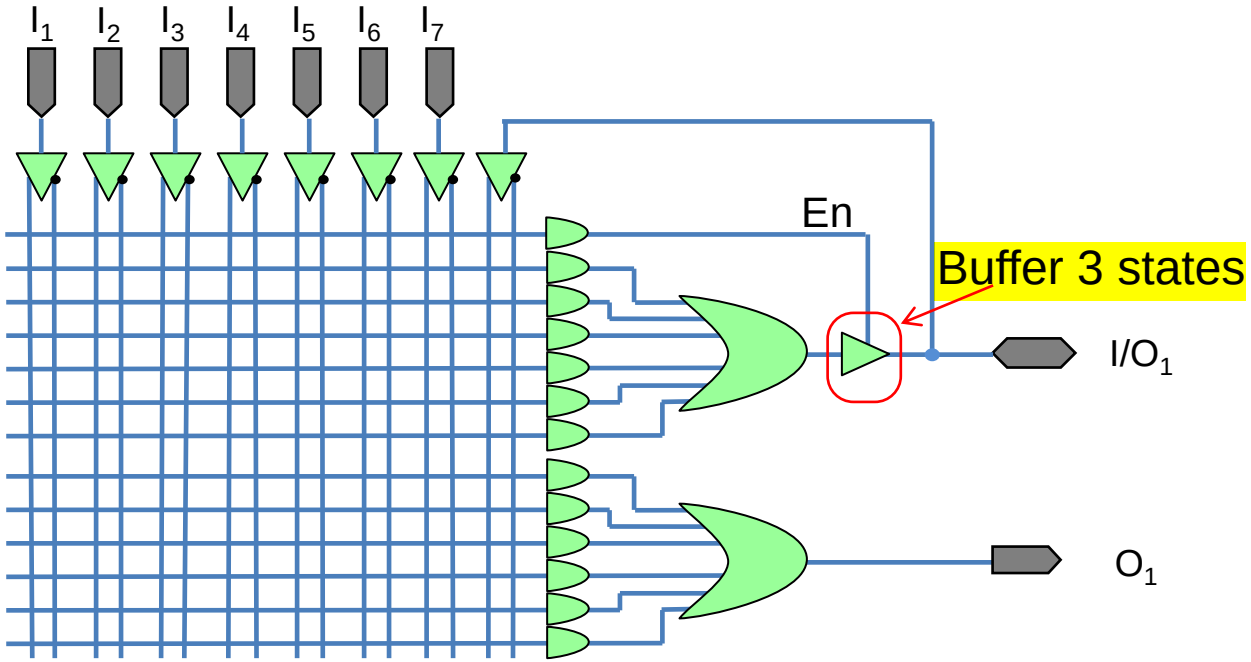
$$Out2 = Out1 + I_3 \bullet I_2 \bullet \overline{I_1}$$

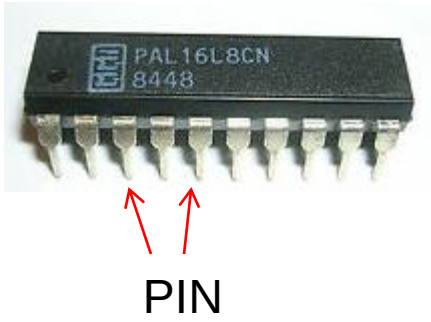
Out1 synthesizes the first six ‘1s’ of the table. Out1 is routed back in the AND matrix and added to the last ‘1’.

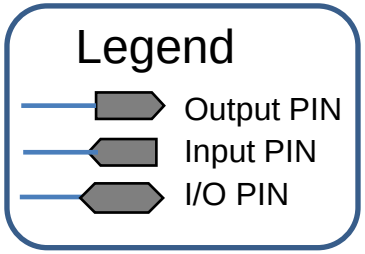
This is a 4-level function implementation => more delay

- PAL: towards commercial devices -

- Programmable Input/Output (I/O)





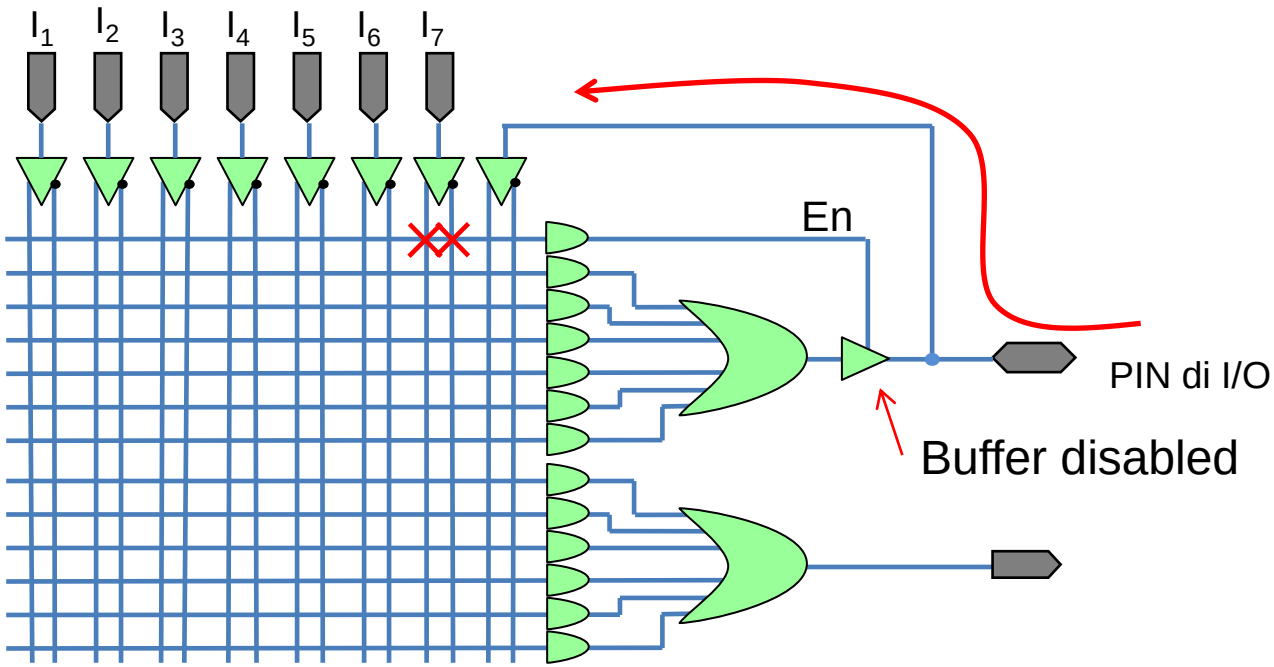


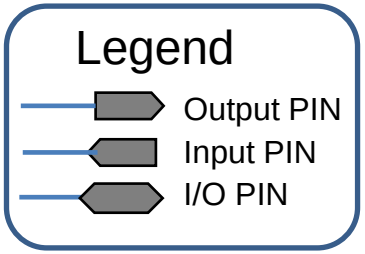
The external connections of a devices can be dedicated inputs (e.g. I_1 – I_7), dedicated outputs (O_1), or programmable Input/Output (I/O_1).

When the buffer 3 state is active (En = 1) the pin is an output, otherwise (En = 0) it is an Input. The En signal can be programmed from the AND matrix

- PAL: Programmable Array Logic -

- Programmable Input/Output (I/O)

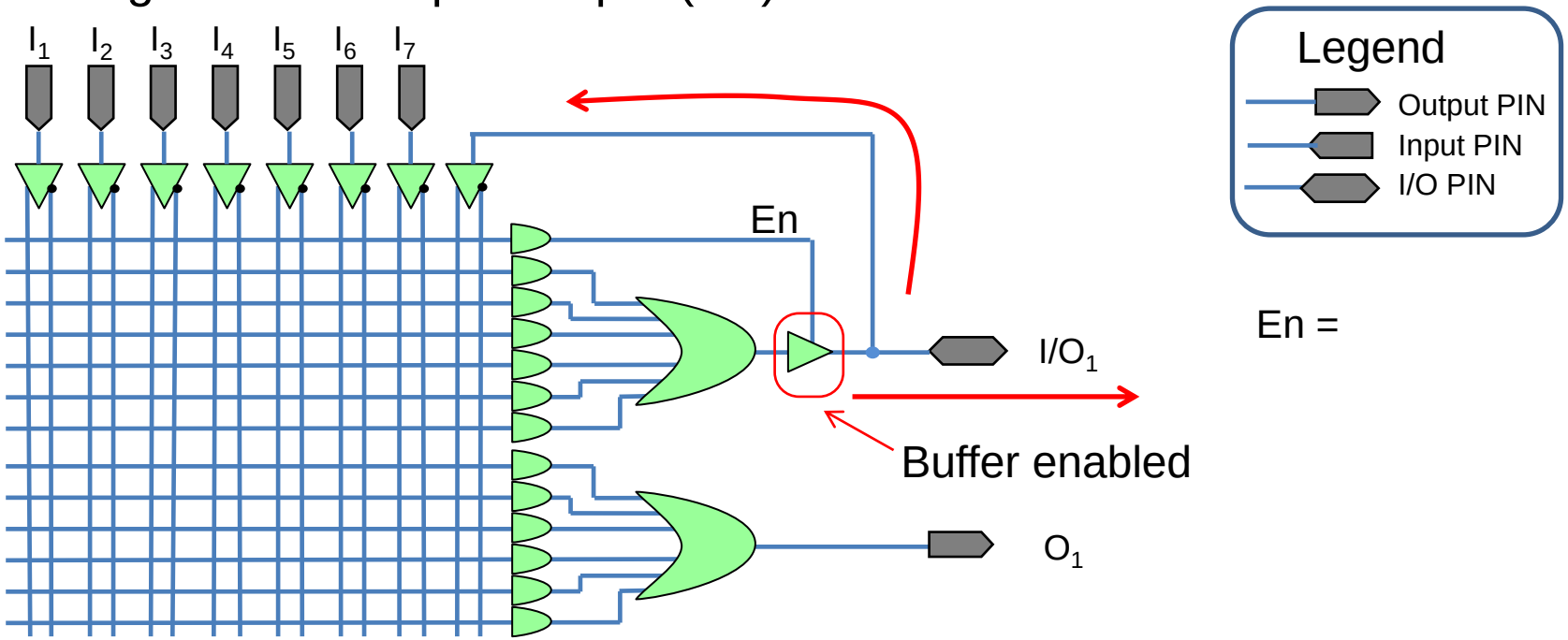




$I_7 \bullet \overline{I_7}$ = En = 0, thus the buffer is disabled and the PIN is an input, which feeds the AND matrix like the other inputs. In this case the logic connected to the disabled buffer can't be used and is wasted.

- PAL: Programmable Array Logic -

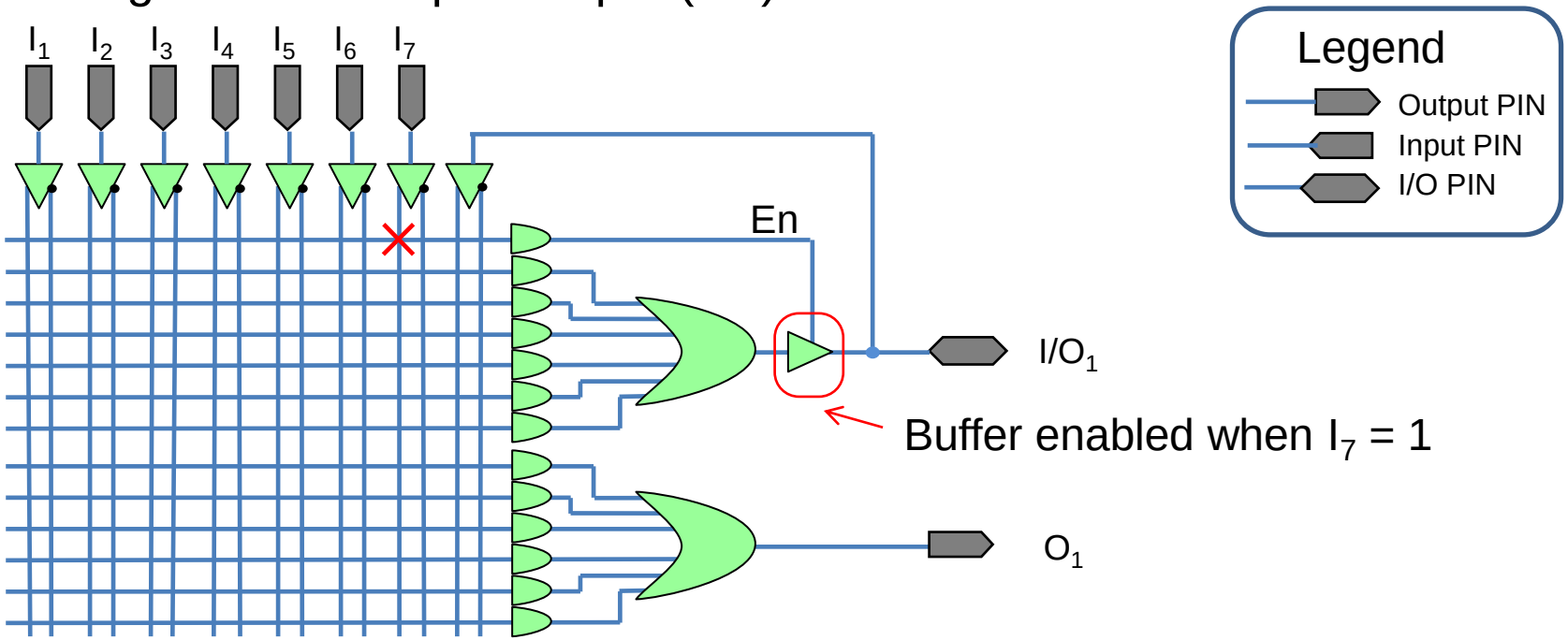
- Programmable Input/Output (I/O)



When the AND connected to En has not connections (no crosses) En = ‘1’ (pull-up on the line) and the PIN is an output. Note that the output signal has a feedback into the AND matrix, and can be re-used in the logic

- PAL: Programmable Array Logic -

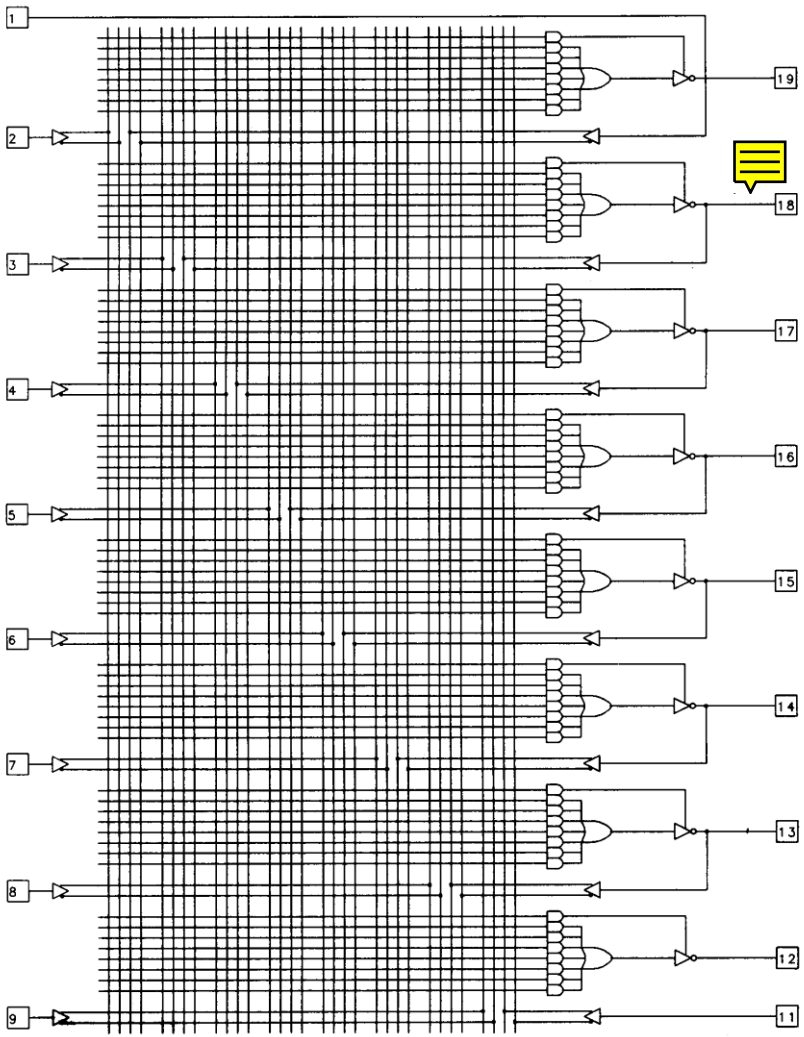
- Programmable Input/Output (I/O)



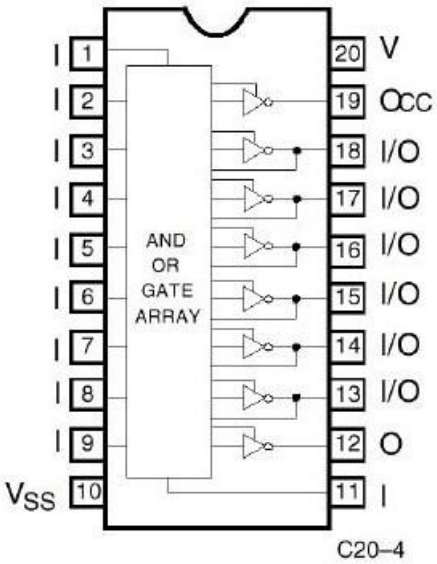
In this example En = I_7 and I_7 works as an output enable.

More complex configurations are possible with a suitable programming

- Commercial Example: PAL 16L8 -



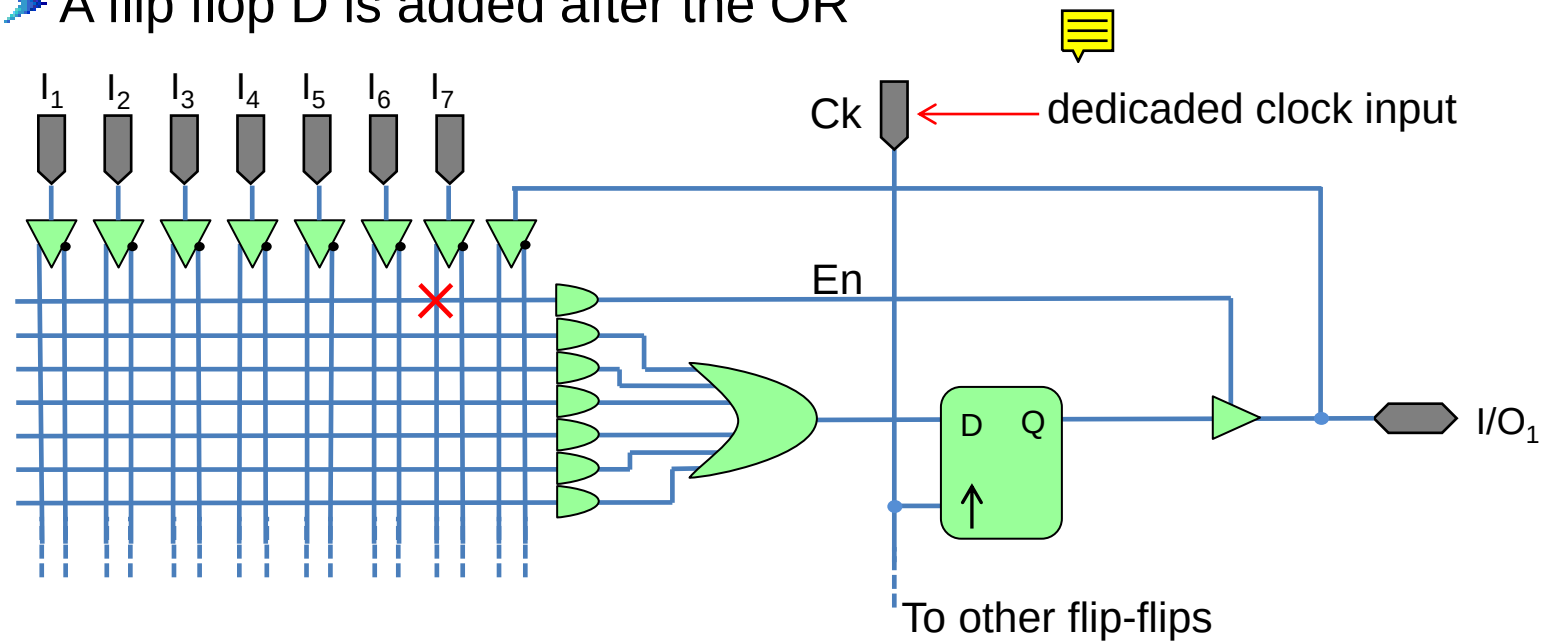
PAL 16L8



PAL with
- 10 dedicated input (I)
- 2 dedicated output (O) with 3 states
- 6 pin switchable between I or O (I/O)

- Registered Programmable Logic -

- A flip flop D is added after the OR

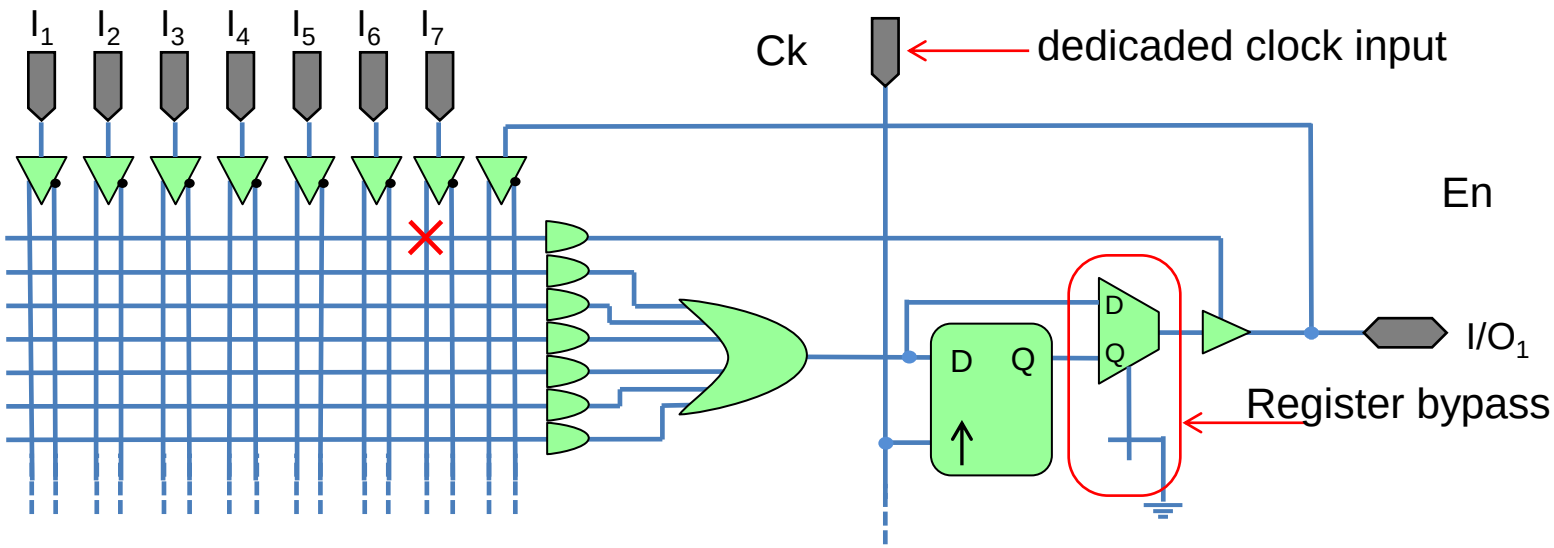


The clock is fed by a dedicaded input to all the flip flops of the devices

- Registered Programmable Logic -

- Register by-passas

A programmable mux can be added to bypass (or not) the register

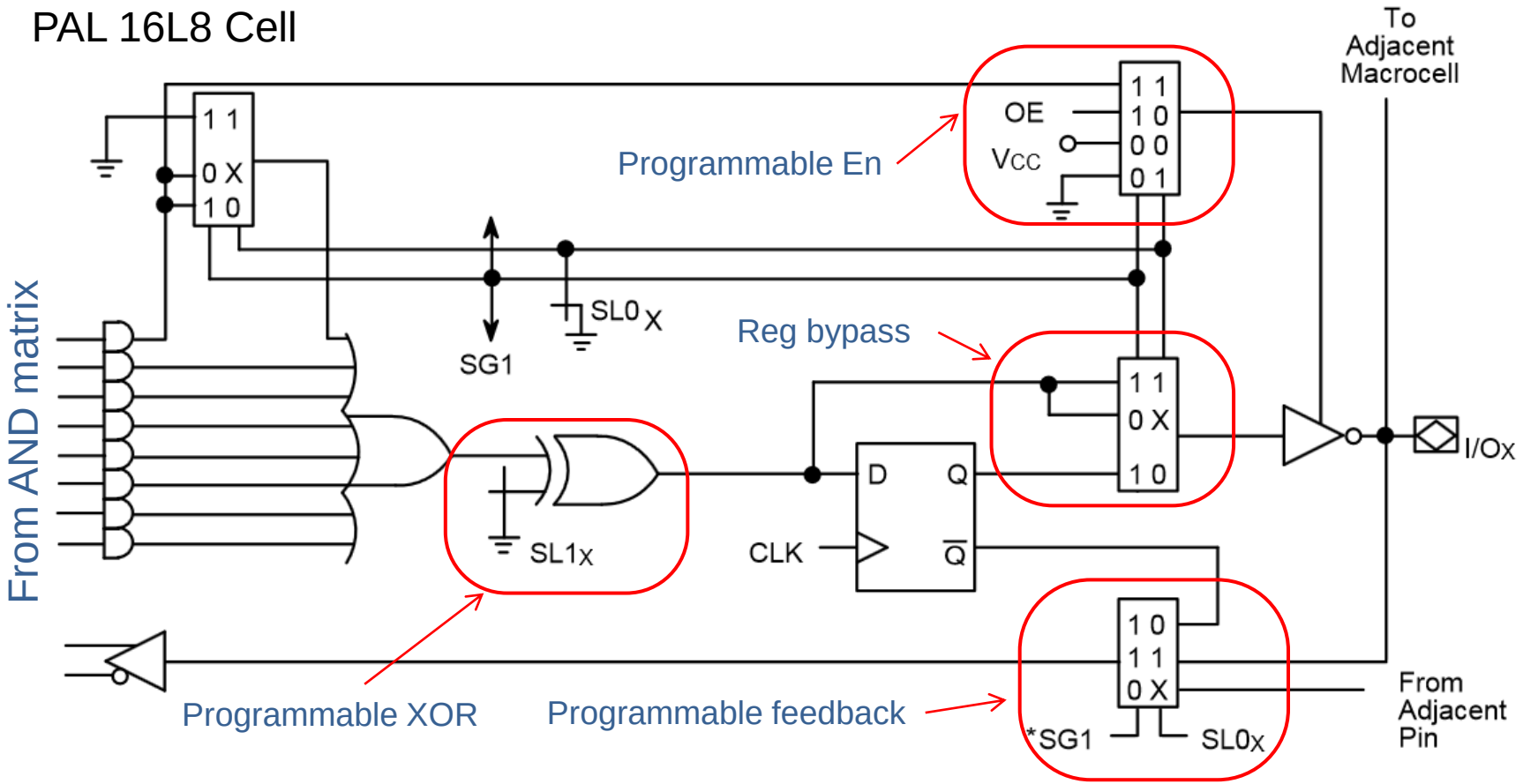


The mux can be programmed to select the output from the register (Q) or bypass the register (D).

The cell synthetizes sequential or combinatorial logic, depending on how the mux is programmed.

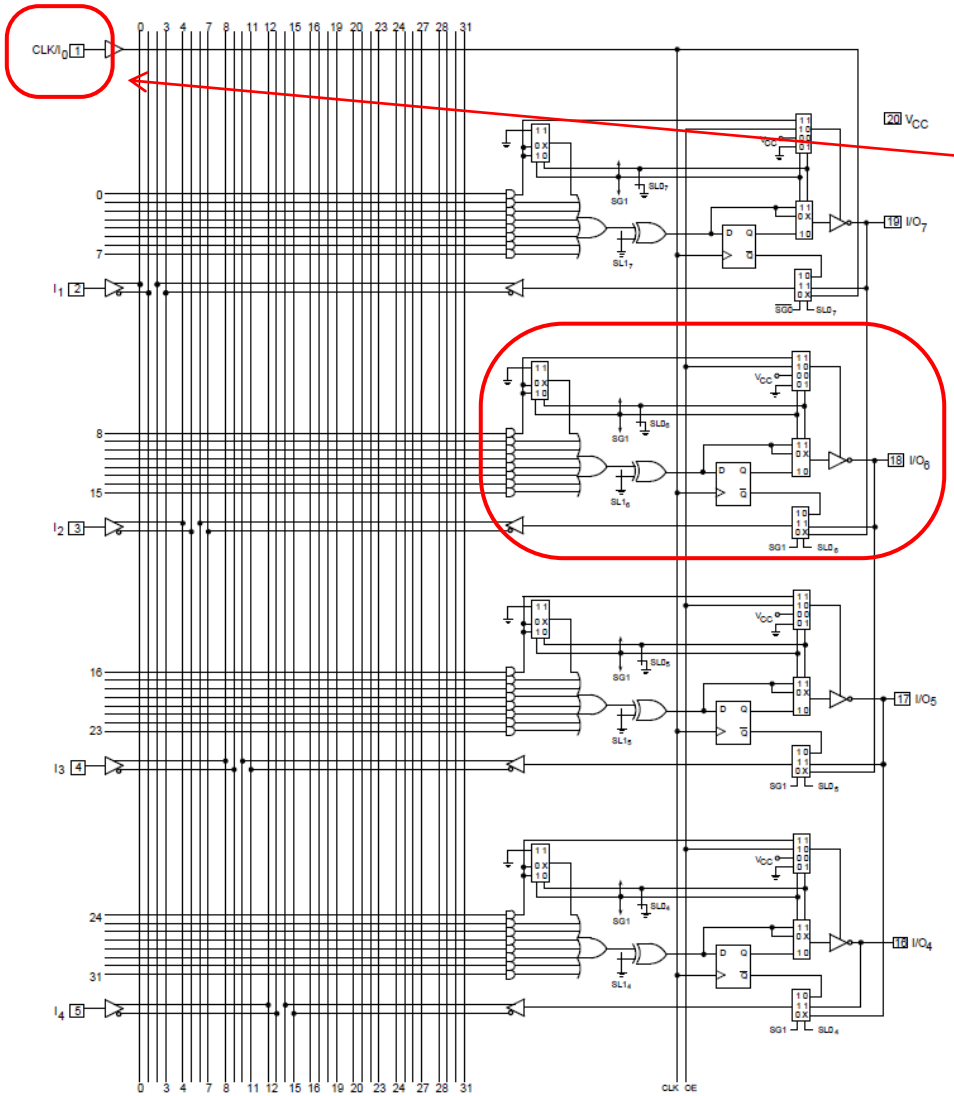
- Commercial Example: PAL 16V8 -

PAL 16L8 Cell



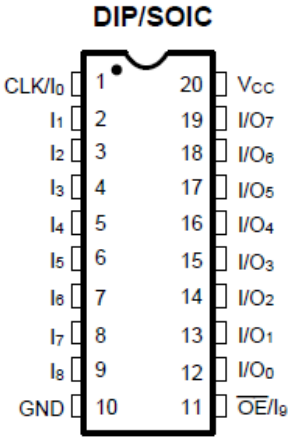
The behaviour of the cell is programmed by the SL0,SG1,SL1 bits

- Commercial Example: PAL 16V8 -



The clk is common to all of the registers

cell

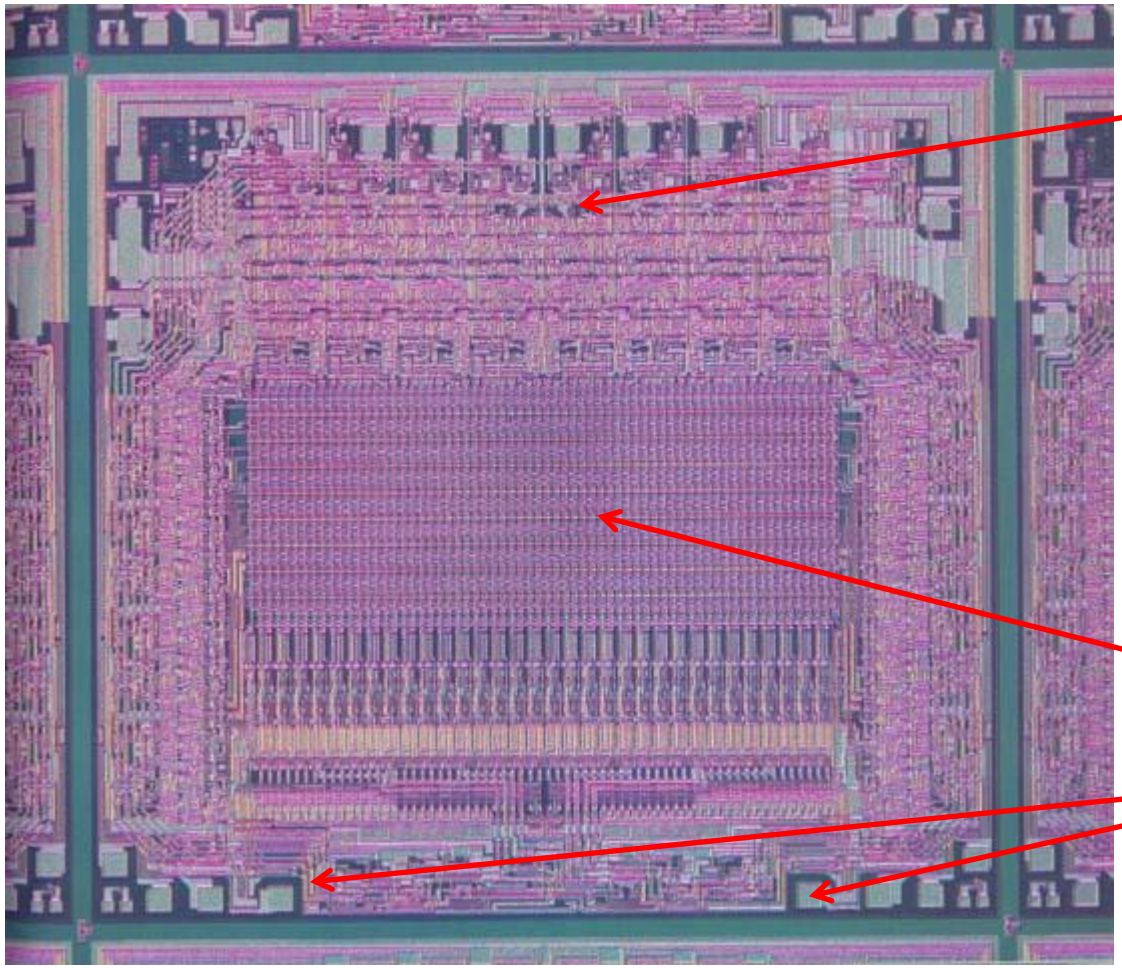


The PAL 16V8 has 8 cells

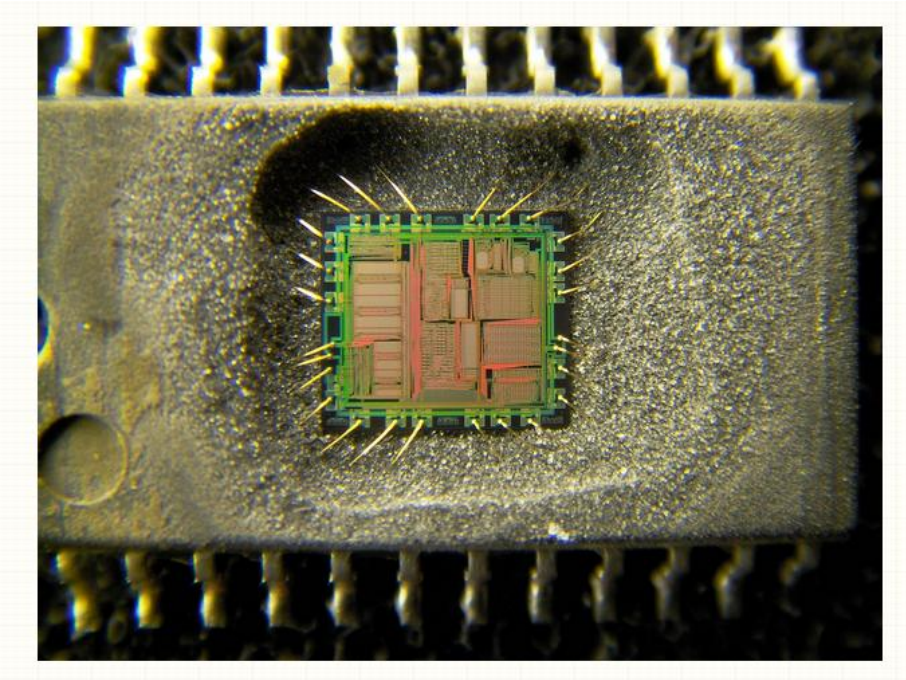
Other cells below…

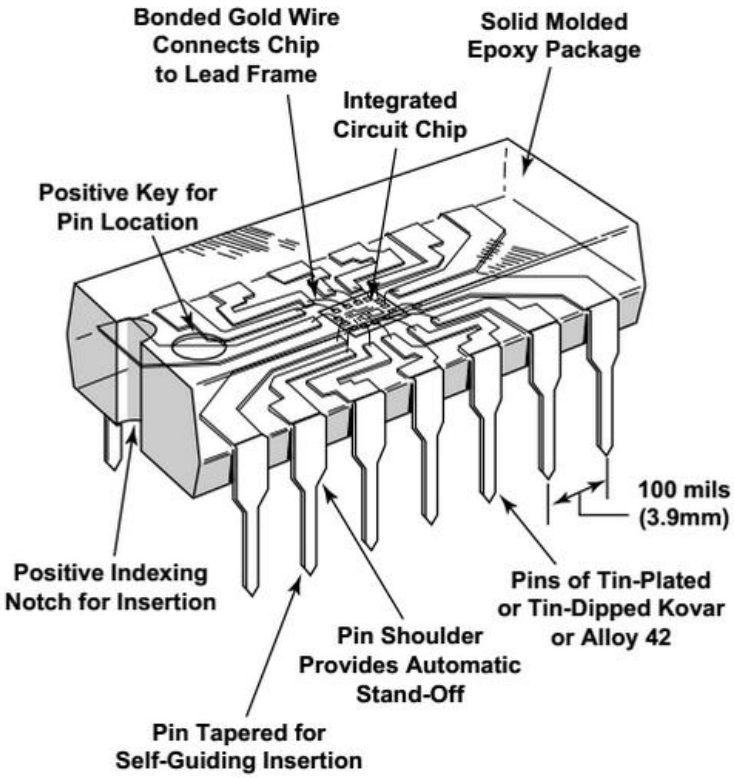
- Commercial Example -

DIE of PAL 16L8



- Commercial Example -



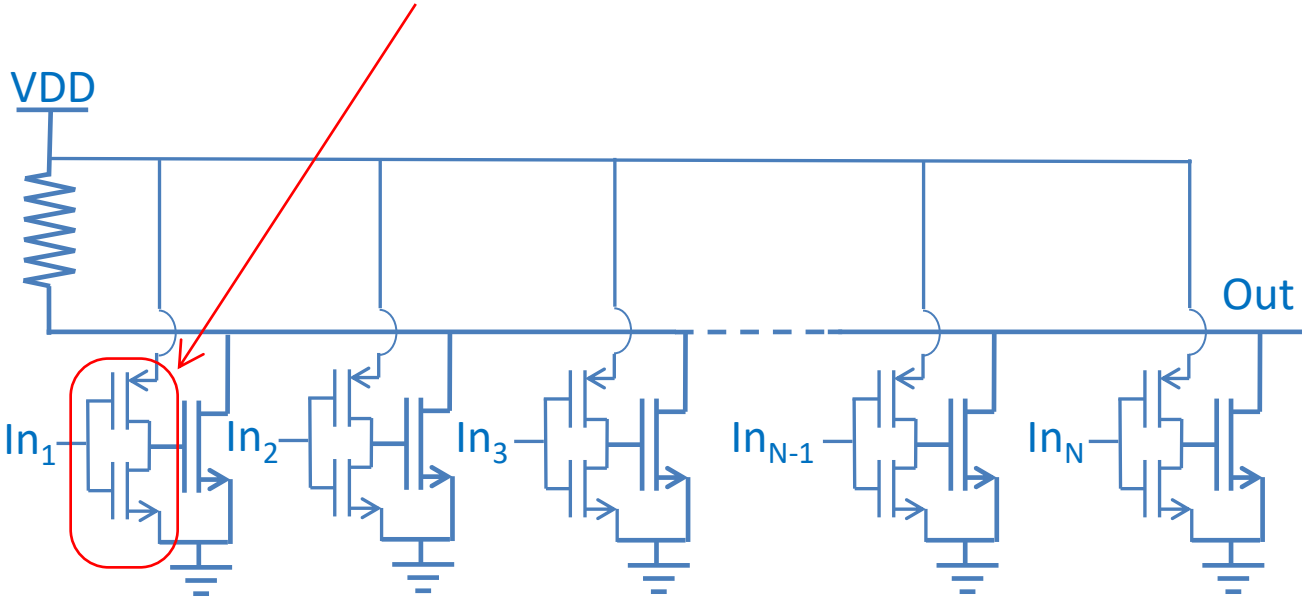


- Summary -

- Wired logic is useful to obtain logic functions with several inputs
- Fuses or FAMOS allows programmability
- Programmable AND matrix followed by fixed OR allows the realization of programmable 2-level logic functions
- Several improvements (feedback, XOR, etc.) can be added
- A register can be present for sequential logic implementation
- PAL devices are (were) commercially available which implement such functions

- Wired Logic -

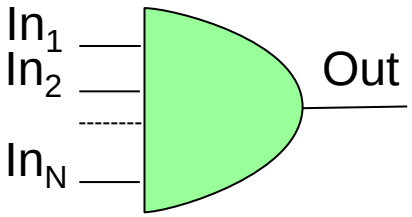
- If we add an inverter on each input we have a wired AND



AND

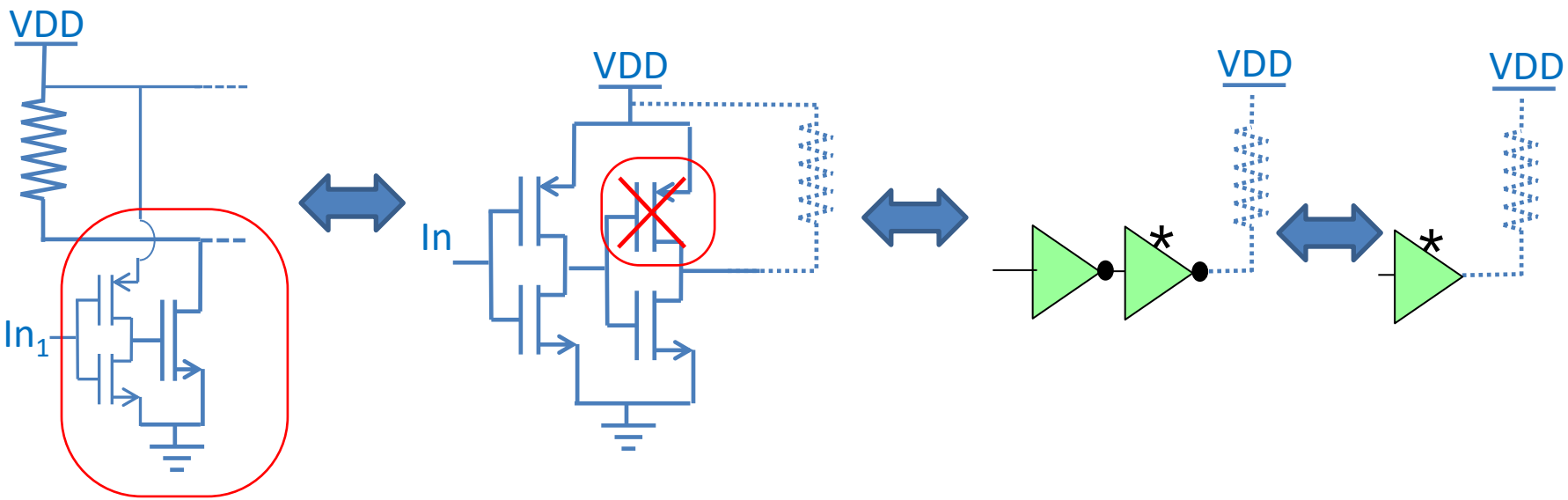
In_1	In_2	...	In_N	Out
0	0	...	0	0
0	0	...	1	0
0	1	...	0	0
0	1	...	1	0
1	0	...	0	0
1	0	...	1	0
1	1	...	0	0
1	1	...	1	1

Out is HIGH only if ALL of the N In are HIGH (Principal Mosfet Off). This is a AND gate with N inputs



- Open Drain/Collector Gates -

- the cell used in the Wired AND is an «open drain» gate



- The «open drain gates are obtained by removing the «high» MOS from the standard gate. Doing so, the gate can only drive low.
- Open drain gates are labelled with a star or similar symbol